

SEARCHING ALGORITHMS

A comprehensive guide to searching algorithms for SDE placements, online assessments, and competitive programming.

1. LINEAR SEARCH

1. Overview

Linear search is the simplest searching algorithm. It sequentially checks every element in a collection until the target is found or the collection ends. It works on any data structure that supports sequential access — arrays, linked lists, vectors, etc.

No preprocessing is required. No ordering assumptions are made. It just walks through each element and compares.

2. Intuition

Simple explanation: Imagine you have a deck of unsorted cards and you are looking for the Ace of Spades. You flip each card one by one, left to right, until you find it. That is linear search.

Analogy: Finding a specific book on a shelf where books are not arranged in any order. You scan from left to right, checking every book until you find the one you need.

Step-by-step reasoning:

1. Start at the first element (index 0).
2. Compare the current element with the target.
3. If they match, return the index (success).
4. If not, move to the next element.
5. Repeat until the target is found or the end is reached.
6. If the end is reached without finding the target, return -1 (failure).

Why it works: The algorithm relies on the pigeonhole principle — if the target exists in the array, it must be at some position. By checking every position exactly once, we guarantee we will find it if it exists. The algorithm is correct because it is exhaustive.

3. When to Use It

- **Unsorted data** — when the array is not sorted and you cannot afford to sort it first.
- **Small datasets** — for very small arrays ($n \leq 100$), the overhead of more complex algorithms is not worth it.
- **Single search** — when you need to search only once; preprocessing for binary search would be wasteful.
- **Linked lists** — binary search is impossible on singly linked lists (no random access); linear search is the only option.

- **Streaming data** — when data arrives in a stream and you can only access it sequentially.
- **Finding first/last occurrence in unsorted data** — when you need the first match.
- **Trivial implementation needed** — in quick scripts or when simplicity is valued over performance.

Common trigger phrases: "unsorted array", "find if element exists", "first occurrence", "linear scan", "brute force", " $O(n)$ search".

4. When Not to Use It

- **Large datasets with multiple searches** — $O(n)$ per search is too slow. Sort once and use binary search ($O(\log n)$).
- **Sorted arrays** — you are wasting the sorted property. Binary search is exponentially faster.
- **Performance-critical code with large n** — linear search on 10^7 elements is 10^7 comparisons. Binary search needs ~ 24 .
- **Real-time systems** — unpredictable worst-case time (n comparisons) can cause missed deadlines.
- **When random access is available and data is sorted** — binary search is strictly better.

5. Core Concepts

Sequential access: The algorithm reads elements one after another. It does not need random access (`arr[i]`), but works with it too.

Comparison: The core operation. Each element is compared to the target. In the worst case, n comparisons are made.

Early termination: If the target is found, the search stops immediately. This gives $O(k)$ time where k is the position of the target.

Sentinel linear search: A small optimization — place the target at the end of the array (as a sentinel) so you never need to check for array bounds inside the loop. This reduces one comparison per iteration.

Probabilistic analysis: If the target is equally likely to be at any position, the average number of comparisons is $(n+1)/2 \approx n/2$.

6. Step-by-Step Algorithm

```
Input:  arr[0..n-1], target
Output: index of target, or -1 if not found

1. FOR i = 0 TO n-1:
2.     IF arr[i] == target:
3.         RETURN i
4. RETURN -1
```

With sentinel:

```
1. last = arr[n-1]
2. arr[n-1] = target  // place sentinel
```

```
3. i = 0
4. WHILE arr[i] != target:
5.     i++
6. arr[n-1] = last      // restore original value
7. IF i < n-1 OR arr[n-1] == target:
8.     RETURN i
9. RETURN -1
```

7. Dry Run

Input: arr = [4, 2, 9, 1, 7, 5], target = 7

Step	i	arr[i]	arr[i] == 7?	Action
1	0	4	No	Continue
2	1	2	No	Continue
3	2	9	No	Continue
4	3	1	No	Continue
5	4	7	Yes	Return 4

Result: Index 4

Input: arr = [4, 2, 9, 1, 7, 5], target = 3

Step	i	arr[i]	arr[i] == 3?	Action
1	0	4	No	Continue
2	1	2	No	Continue
3	2	9	No	Continue
4	3	1	No	Continue
5	4	7	No	Continue
6	5	5	No	Continue
7	-	-	-	End of array

Result: -1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Basic linear search
int linearSearch(const vector<int>& arr, int target) {
    int n = arr.size();
```

```
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i; // Found: return index
        }
    }
    return -1; // Not found
}

// Sentinel linear search (slightly faster, avoids bounds check)
int sentinelLinearSearch(vector<int>& arr, int target) {
    int n = arr.size();
    if (n == 0) return -1;

    int last = arr[n - 1];
    arr[n - 1] = target; // Place sentinel

    int i = 0;
    while (arr[i] != target) {
        i++;
    }

    arr[n - 1] = last; // Restore

    if (i < n - 1 || arr[n - 1] == target) {
        return i;
    }
    return -1;
}

// Find all occurrences
vector<int> linearSearchAll(const vector<int>& arr, int target) {
    vector<int> indices;
    for (int i = 0; i < (int)arr.size(); i++) {
        if (arr[i] == target) {
            indices.push_back(i);
        }
    }
    return indices;
}

// Example usage
int main() {
    vector<int> arr = {4, 2, 9, 1, 7, 5, 7};
    int target = 7;

    int idx = linearSearch(arr, target);
    cout << "Index of " << target << ": " << idx << endl; // 4

    vector<int> all = linearSearchAll(arr, target);
    cout << "All occurrences: ";
    for (int i : all) cout << i << " "; // 4 6
    cout << endl;
}
```

```
    return 0;
}
```

9. Python Implementation

```
from typing import List

# Basic linear search
def linear_search(arr: List[int], target: int) -> int:
    for i in range(len(arr)):
        if arr[i] == target:
            return i # Found: return index
    return -1 # Not found

# Sentinel linear search
def sentinel_linear_search(arr: List[int], target: int) -> int:
    n = len(arr)
    if n == 0:
        return -1

    last = arr[-1]
    arr[-1] = target # Place sentinel

    i = 0
    while arr[i] != target:
        i += 1

    arr[-1] = last # Restore

    if i < n - 1 or arr[-1] == target:
        return i
    return -1

# Find all occurrences
def linear_search_all(arr: List[int], target: int) -> List[int]:
    return [i for i, x in enumerate(arr) if x == target]

# Example usage
if __name__ == "__main__":
    arr = [4, 2, 9, 1, 7, 5, 7]
    target = 7

    idx = linear_search(arr, target)
    print(f"Index of {target}: {idx}") # 4

    all_occ = linear_search_all(arr, target)
    print(f"All occurrences: {all_occ}") # [4, 6]
```

10. Code Explanation

Basic version:

- Loop from `i = 0` to `n-1`.
- At each index, compare `arr[i]` with `target`.
- On match, return `i` immediately.
- If loop ends, return `-1`.

Sentinel version:

- Save the last element, then overwrite it with `target`.
- Loop without bounds check — sentinel guarantees termination.
- After loop, restore the last element.
- Check if we stopped before the last element, or if the last element itself was the target.

Multiple occurrences:

- Same loop, but instead of returning on first match, collect all matching indices into a vector.

Edge cases handled:

- Empty array: loop does not execute, returns `-1`.
- Target at first position: returns `0` immediately.
- Target not present: returns `-1` after full scan.
- Duplicates: basic version returns first match; `linearSearchAll` returns all.

11. Complexity Analysis

Case	Time	Space
Best case	$O(1)$	$O(1)$
Worst case	$O(n)$	$O(1)$
Average case	$O(n)$	$O(1)$
Sentinel version	$O(n)$	$O(1)$

- **Best case:** target is at index `0`.
- **Worst case:** target is at last index or not present.
- **Average case:** $(n+1)/2$ comparisons if target is present.
- **Space:** $O(1)$ extra space (iterative). No recursion.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Find first occurrence	Unsorted array, "first index where"	Linear scan, return on first match	First occurrence in unsorted array
Find all occurrences	"All indices where", "count occurrences"	Collect all matches in a vector	Find all positions of target

Pattern	How to Identify	Approach	Example Problems
Find max/min	"Find the largest/smallest"	Track max/min during scan	Maximum element in array
Two-pointer scan	"Find pair with sum X"	One pointer at start, one at end (or both at start)	Two Sum (unsorted brute force)
Linear search in 2D	"Search in matrix"	Nested loop over rows and columns	Search in unsorted matrix

13. Common Mistakes

- **Forgetting to return -1** after the loop when the target is not found.
- **Starting from index 1** instead of 0 (off-by-one).
- **Off-by-one in loop condition**: writing `i <= n` instead of `i < n` causes array out-of-bounds.
- **Modifying the array** in sentinel search and forgetting to restore the original value.
- **Using linear search on sorted data** — much slower than binary search for large n .
- **Not handling empty array** — loop may still execute with `i < 0` if using wrong condition.
- **Returning the value instead of the index** — most problems ask for the index.

14. Edge Cases

- Empty array: `[]`, search for any target $\rightarrow$ -1.
- Single element: `[5]`, search for 5 $\rightarrow$ 0; search for 3 $\rightarrow$ -1.
- Target at first position: returns 0 immediately.
- Target at last position: scans all elements, returns $n-1$.
- Target not present: scans all elements, returns -1.
- Multiple occurrences: basic version returns first.
- All elements equal to target: returns 0 (first position).
- Null or invalid input: should handle gracefully.

15. Variations

Variation	What Changes	When Used	Importance
Sentinel linear search	Places target at end to remove bounds check	Performance optimization for large arrays	Low (interview novelty)
Recursive linear search	Uses recursion instead of loop	Academic, not practical	Low
Parallel linear search	Divides array among threads	Very large datasets, multi-core	Low (CP not needed)
Probabilistic search (randomized)	Picks random indices	Expected to find early for frequent targets	Low

16. Related Algorithms

Algorithm	Connection	How to Choose
Binary search	Both search; binary requires sorted data	Sorted → binary; unsorted → linear
Jump search	Generalization of linear search with blocks	Sorted + need better than $O(n)$ but simpler than binary
Exponential search	Starts with small range, then binary search	Unbounded/infinite arrays
Hashing	$O(1)$ search with hash table	Multiple searches, unsorted, space not an issue

Decision rule: If data is sorted → binary search. If data is unsorted and you search once → linear search. If data is unsorted and you search many times → hash table (or sort + binary search).

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Linear Search	GFG	Basic linear search	Easy
Search an Element in an Array	LeetCode	Find index of element	Easy

Medium

Problem	Platform	Pattern	Difficulty
Count Occurrences of an Element	GFG	Count all occurrences	Medium
Find the Maximum Element	GFG	Linear scan for max	Medium

Hard

Problem	Platform	Pattern	Difficulty
Find the Missing Number in Arithmetic Progression	LeetCode	Linear scan with difference check	Medium-Hard
Find the Celebrity	LeetCode	Linear elimination	Hard

18. Interview Explanation

"Linear search is the simplest searching algorithm. We iterate through the array from index 0 to $n-1$, comparing each element with the target. If we find a match, we return the index. If we reach the end without finding it, we return -1 . It has $O(n)$ time complexity in the worst case and $O(1)$ space. It works on any data — sorted or unsorted — but it's only practical for small datasets, unsorted data, or when we only need to search once. For sorted data or multiple searches, we would use binary search or a hash table instead."

19. Revision Notes

- **Key idea:** Exhaustive scan — check every element.
- **Works on:** Any iterable collection (arrays, lists, linked lists).
- **No preprocessing required.**
- **Complexity:** $O(n)$ time, $O(1)$ space.
- **Best case:** $O(1)$ — target at index 0.
- **Worst case:** $O(n)$ — target at end or not present.
- **Common trap:** Forgetting to return -1 after the loop.
- **Optimization:** Sentinel search removes one comparison per iteration.
- **Use when:** Unsorted data, single search, small arrays, linked lists.
- **Don't use when:** Sorted data, many searches, large n .

20. Final Cheat Sheet

Aspect	Detail
When to use	Unsorted data, single search, small n , linked lists
Main operation	Compare each element with target
Time	$O(n)$ worst, $O(1)$ best
Space	$O(1)$
Key code idea	<pre>for (int i = 0; i < n; i++) if (arr[i] == target) return i;</pre>
Edge cases	Empty array, single element, target not found, duplicates
Return value	Index of first match, or -1

2. BINARY SEARCH

1. Overview

Binary search is an efficient algorithm for finding a target value in a **sorted** array. It repeatedly divides the search space in half by comparing the target with the middle element. With each comparison, it eliminates half of the remaining elements, giving $O(\log n)$ time complexity.

It is one of the most fundamental and frequently used algorithms in programming.

2. Intuition

Simple explanation: You have a sorted phone book and need to find "Smith". You do not start from page 1. You open the book in the middle. If "Smith" comes after the current page alphabetically, you discard the first half and repeat on the second half. Each step cuts the search space in half.

Analogy: The "guess the number" game where someone thinks of a number between 1 and 100, and you guess. You say "50". They say "higher". Now you know the number is between 51 and 100. You guess "75". This is binary search.

Step-by-step reasoning:

1. Look at the middle element of the sorted array.
2. If it matches the target — done.
3. If the target is smaller than the middle, the target must be in the left half (since the array is sorted).
4. If the target is larger, it must be in the right half.
5. Repeat on the chosen half until the target is found or the search space is empty.

Why it works: The sorted order is the key property. Because the array is sorted, comparing with the middle element tells us conclusively which half the target must be in. This monotonic property (the "sortedness") allows us to discard half the search space. This is the divide-and-conquer principle.

3. When to Use It

- **Sorted array** — this is the primary requirement.
- **Multiple searches on the same data** — sort once, search many times.
- **Finding boundaries** — first/last occurrence of a value, lower/upper bound.
- **Finding a value in a monotonic function** — search on answer, binary search on the solution space.
- **Large datasets** — $O(\log n)$ is exponentially faster than $O(n)$.
- **Real-time systems** — guaranteed $O(\log n)$ worst case.
- **When the search space is a range of integers** — "find the smallest k such that $f(k)$ is true".

Common trigger phrases: "sorted array", "sorted list", "non-decreasing", "non-increasing", "monotonic", "first occurrence", "last occurrence", "find in $O(\log n)$ ", "search in sorted", "minimum k such that", "maximum k such that".

4. When Not to Use It

- **Unsorted data** — binary search requires sorted data. Either sort first ($O(n \log n)$) or use linear search.
- **Linked lists** — no random access $O(1)$ in linked lists. Binary search becomes $O(n \log n)$ or worse.
- **Very small arrays** ($n < 10$) — linear search may be faster due to cache locality and simpler code.
- **Single search on unsorted data** — sorting costs $O(n \log n)$, which is worse than $O(n)$ linear search.
- **Data with frequent insertions/deletions** — maintaining sorted order has overhead. Consider a balanced BST or hash table.
- **Non-comparable data** — you need a total order (elements must be comparable with $<$, $>$, $=$).

5. Core Concepts

Sorted array: The fundamental requirement. The array must be sorted in non-decreasing (or non-increasing) order for standard binary search.

Search space: The range of indices being considered. Initially $[0, n-1]$. Each iteration halves it.

Midpoint calculation: $mid = low + (high - low) / 2$. This avoids integer overflow (unlike $(low + high) / 2$).

Three-way comparison: Compare target with $arr[mid]$. Three outcomes: less than, equal to, greater than.

Invariant: The target, if present, is always in the range $[low, high]$. This property must hold throughout the algorithm.

Termination: When $low > high$ (standard version) or when the target is found.

Monotonic predicate: Binary search can be generalized to find the first index where a predicate $P(i)$ is true, given that P is false for all $i < \text{some threshold}$ and true for all $i \geq \text{threshold}$.

6. Step-by-Step Algorithm

Standard binary search (find exact target):

```
Input:  sorted arr[0..n-1], target
Output: index of target, or -1 if not found

1. low = 0, high = n - 1
2. WHILE low <= high:
3.     mid = low + (high - low) / 2
4.     IF arr[mid] == target:
5.         RETURN mid
6.     ELSE IF arr[mid] < target:
7.         low = mid + 1      // search right half
8.     ELSE:
9.         high = mid - 1    // search left half
10. RETURN -1
```

Binary search for first occurrence (lower bound):

```
1. low = 0, high = n - 1
2. ans = -1
3. WHILE low <= high:
4.     mid = low + (high - low) / 2
5.     IF arr[mid] == target:
6.         ans = mid        // record answer
7.         high = mid - 1   // continue searching left
8.     ELSE IF arr[mid] < target:
9.         low = mid + 1
10.    ELSE:
11.        high = mid - 1
12. RETURN ans
```

7. Dry Run

Input: arr = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72], target = 23

Step	low	high	mid	arr[mid]	Comparison	Action
1	0	9	4	16	$16 < 23$	low = 5
2	5	9	7	45	$45 > 23$	high = 6
3	5	6	5	23	$23 == 23$	Return 5

Result: Index 5

Input: arr = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72], target = 10

Step	low	high	mid	arr[mid]	Comparison	Action
1	0	9	4	16	16 > 10	high = 3
2	0	3	1	5	5 < 10	low = 2
3	2	3	2	8	8 < 10	low = 3
4	3	3	3	12	12 > 10	high = 2
5	3	2	-	-	low > high	Return -1

Result: -1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Standard binary search (exact match)
int binarySearch(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2; // Avoid overflow

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1; // Target is in right half
        } else {
            high = mid - 1; // Target is in left half
        }
    }
    return -1; // Not found
}

// First occurrence (lower bound)
int firstOccurrence(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size() - 1;
    int ans = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            ans = mid;
            high = mid - 1; // Continue searching left
        } else if (arr[mid] < target) {
```

```

        low = mid + 1;
    } else {
        high = mid - 1;
    }
}
return ans;
}

// Last occurrence (upper bound style)
int lastOccurrence(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size() - 1;
    int ans = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            ans = mid;
            low = mid + 1; // Continue searching right
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}

// Recursive version
int binarySearchRecursive(const vector<int>& arr, int target, int low, int high) {
    if (low > high) return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return binarySearchRecursive(arr, target, mid + 1,
high);
    return binarySearchRecursive(arr, target, low, mid - 1);
}

// Example usage
int main() {
    vector<int> arr = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72};
    int target = 23;

    int idx = binarySearch(arr, target);
    cout << "Index of " << target << ": " << idx << endl; // 5

    // With duplicates
    vector<int> arr2 = {1, 3, 3, 3, 5, 7, 9};
    cout << "First occurrence of 3: " << firstOccurrence(arr2, 3) << endl; // 1
    cout << "Last occurrence of 3: " << lastOccurrence(arr2, 3) << endl; // 3
}

```

```
    return 0;
}
```

9. Python Implementation

```
from typing import List

# Standard binary search
def binary_search(arr: List[int], target: int) -> int:
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2 # Avoid overflow

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# First occurrence
def first_occurrence(arr: List[int], target: int) -> int:
    low, high = 0, len(arr) - 1
    ans = -1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == target:
            ans = mid
            high = mid - 1
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return ans

# Last occurrence
def last_occurrence(arr: List[int], target: int) -> int:
    low, high = 0, len(arr) - 1
    ans = -1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == target:
            ans = mid
```

```

        low = mid + 1
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1

    return ans

# Recursive version
def binary_search_recursive(arr: List[int], target: int, low: int, high: int) ->
int:
    if low > high:
        return -1

    mid = low + (high - low) // 2

    if arr[mid] == target:
        return mid
    if arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, high)
    return binary_search_recursive(arr, target, low, mid - 1)

# Example usage
if __name__ == "__main__":
    arr = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72]
    target = 23

    idx = binary_search(arr, target)
    print(f"Index of {target}: {idx}") # 5

# With duplicates
arr2 = [1, 3, 3, 3, 5, 7, 9]
print(f"First occurrence of 3: {first_occurrence(arr2, 3)}") # 1
print(f>Last occurrence of 3: {last_occurrence(arr2, 3)}") # 3

```

10. Code Explanation

Standard binary search:

- `low = 0`, `high = n-1` — define the search space.
- `while (low <= high)` — continue while the search space is non-empty.
- `mid = low + (high - low) / 2` — compute midpoint safely (avoids overflow of `low + high`).
- Three branches:
 - `arr[mid] == target`: exact match found, return `mid`.
 - `arr[mid] < target`: target is in the right half, set `low = mid + 1`.
 - `arr[mid] > target`: target is in the left half, set `high = mid - 1`.
- If loop exits without finding target, return `-1`.

First occurrence:

- Same as standard, but on finding a match, we record the index and continue searching the left half ($high = mid - 1$).
- This ensures we find the leftmost occurrence.

Last occurrence:

- On finding a match, record the index and continue searching the right half ($low = mid + 1$).
- This ensures we find the rightmost occurrence.

Edge cases handled:

- Empty array: $high = -1$, loop does not execute, returns -1.
- Single element: works correctly.
- Target not present: returns -1.
- Duplicates: first/last occurrence variants handle correctly.

11. Complexity Analysis

Case	Time	Space
Best case	$O(1)$	$O(1)$
Worst case	$O(\log n)$	$O(1)$
Average case	$O(\log n)$	$O(1)$
Recursive version	$O(\log n)$	$O(\log n)$ (call stack)

- **Best case:** Target is the middle element (first comparison).
- **Worst case:** Target is at a leaf of the decision tree, or not present.
- **Space:** $O(1)$ for iterative version. Recursive version uses $O(\log n)$ stack space.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Exact match	"Find x in sorted array"	Standard binary search	Binary Search (LeetCode 704)
First occurrence	"First index where value = x"	Binary search, move left on match	First Bad Version (LeetCode 278)
Last occurrence	"Last index where value = x"	Binary search, move right on match	Find Last Position in Sorted Array
Lower bound	"First index where value $\geq x$ "	Binary search with predicate	Lower Bound STL
Upper bound	"First index where value $> x$ "	Binary search with predicate	Upper Bound STL
Search in rotated array	"Sorted array rotated at pivot"	Modified binary search with rotation check	Search in Rotated Sorted Array (LC 33)

Pattern	How to Identify	Approach	Example Problems
Search on answer	"Minimize max", "maximize min", "find smallest k such that"	Binary search on the solution space	Capacity To Ship Packages (LC 1011)
Peak finding	"Find peak element in mountain array"	Binary search on derivative	Peak Index in Mountain Array (LC 852)

13. Common Mistakes

- **Overflow in mid calculation:** `mid = (low + high) / 2` can overflow for large ints. Use `low + (high - low) / 2`.
- **Off-by-one in loop condition:** `while (low < high)` vs `while (low <= high)`. The wrong choice causes infinite loops or missed elements.
- **Infinite loop:** When `low` and `high` are adjacent and `mid` keeps pointing to the same element. This happens when updating `low = mid` instead of `low = mid + 1` (or `high = mid` instead of `high = mid - 1`).
- **Not updating low/high correctly:** Forgetting `+1` or `-1` causes infinite loops.
- **Assuming the array is sorted when it is not:** Binary search gives wrong results on unsorted data.
- **Wrong mid for first/last occurrence:** Not handling the `arr[mid] == target` case correctly.
- **Integer overflow in mid:** Already mentioned, but very common.
- **Not handling empty array.**
- **Using `size_t` for low/high** — `size_t` is unsigned; `low - 1` can wrap around.

14. Edge Cases

- Empty array: `[]`, search any $\rightarrow$ -1.
- Single element: `[5]`, search 5 $\rightarrow$ 0; search 3 $\rightarrow$ -1.
- Two elements: `[1, 3]`, search 1 $\rightarrow$ 0; search 3 $\rightarrow$ 1; search 2 $\rightarrow$ -1.
- All equal elements: `[5, 5, 5, 5]`, search 5 $\rightarrow$ first occurrence returns 0, last returns 3.
- Target smaller than all elements: `[10, 20, 30]`, search 5 $\rightarrow$ -1.
- Target larger than all elements: `[10, 20, 30]`, search 40 $\rightarrow$ -1.
- Array with negative numbers: binary search works fine.
- Overflow case: `low = INT_MAX - 1`, `high = INT_MAX` — `(low + high) / 2` overflows.
- Duplicate values: standard binary search returns any occurrence; use first/last variants for specific positions.

15. Variations

Variation	What Changes	When Used	Importance
Binary search on answer	Search space is a range of integers, not an array	Optimization problems, "minimize max", "maximize min"	Very High (placement + CP)
Ternary search	Divides into 3 parts instead of 2	Unimodal functions (finding max/min of convex function)	Medium

Variation	What Changes	When Used	Importance
Binary search on real numbers	Uses precision threshold instead of integer comparison	When answer is a floating point value	High
Exponential + binary search	First find range, then binary search	Unbounded/infinite arrays	Medium
Binary search in 2D matrix	Treat matrix as flattened sorted array	Search in row-wise and column-wise sorted matrix	High

16. Related Algorithms

Algorithm	Connection	How to Choose
Linear search	Both search; binary requires sorted data	Unsorted → linear; sorted → binary
Ternary search	Divides into 3 parts; for unimodal functions	Need to find max/min of unimodal function → ternary
Exponential search	Combines range finding + binary search	Unbounded arrays → exponential
Jump search	Block-based search on sorted array	When binary search is too complex but need better than linear
Interpolation search	Binary search with intelligent mid (probe based on value)	Uniformly distributed data → interpolation
Lower/upper bound	Variants of binary search	Finding insertion position, range queries

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Binary Search	LeetCode 704	Standard binary search	Easy
First Bad Version	LeetCode 278	First occurrence binary search	Easy

Medium

Problem	Platform	Pattern	Difficulty
Search in Rotated Sorted Array	LeetCode 33	Modified binary search on rotated array	Medium
Find First and Last Position of Element in Sorted Array	LeetCode 34	First + last occurrence	Medium

Problem	Platform	Pattern	Difficulty
Capacity To Ship Packages Within D Days	LeetCode 1011	Binary search on answer	Medium

Hard

Problem	Platform	Pattern	Difficulty
Median of Two Sorted Arrays	LeetCode 4	Binary search on two arrays	Hard
Split Array Largest Sum	LeetCode 410	Binary search on answer	Hard
Kth Smallest Element in a Sorted Matrix	LeetCode 378	Binary search on value range	Hard

18. Interview Explanation

"Binary search finds a target in a sorted array in $O(\log n)$ time. We maintain two pointers — low and high — representing the current search range. We compute the midpoint and compare the middle element with the target. If they match, we return the index. If the target is smaller, we search the left half by setting $high = mid - 1$. If larger, we search the right half by setting $low = mid + 1$. We repeat until low exceeds high, which means the target is not present. The key insight is that the sorted property lets us eliminate half the search space with each comparison. The mid calculation must use $low + (high - low) / 2$ to avoid integer overflow. Variations include finding the first or last occurrence of a value, and binary search on the answer space for optimization problems."

19. Revision Notes

- **Key idea:** Divide search space in half at each step.
- **Requirement:** Sorted array (or monotonic predicate).
- **Mid formula:** $mid = low + (high - low) / 2$ (avoid overflow).
- **Loop condition:** $while (low \leq high)$.
- **Updates:** $low = mid + 1$ (right), $high = mid - 1$ (left).
- **Complexity:** $O(\log n)$ time, $O(1)$ space.
- **First occurrence:** On match, $high = mid - 1$ (search left).
- **Last occurrence:** On match, $low = mid + 1$ (search right).
- **Common trap:** Off-by-one in loop condition leads to infinite loop or missed elements.
- **Common trap:** Using $mid = (low + high) / 2$ causes overflow for large ints.
- **Binary search on answer:** Search on integer range, not array. Predicate is monotonic.
- **STL functions:** `lower_bound()`, `upper_bound()`, `binary_search()`.

20. Final Cheat Sheet

Aspect	Detail
When to use	Sorted array, multiple searches, first/last occurrence, search on answer
Main operation	Compare target with middle element, eliminate half

Aspect	Detail
Time	$O(\log n)$
Space	$O(1)$ iterative, $O(\log n)$ recursive
Mid formula	<code>mid = low + (high - low) / 2</code>
Key code	<pre>while (low <= high) { mid = ...; if (arr[mid] == target) return mid; ... }</pre>
Edge cases	Empty array, single element, duplicates, target not present
STL helper	<code>lower_bound</code> , <code>upper_bound</code> , <code>binary_search</code>

3. SEARCH IN ROTATED SORTED ARRAY

1. Overview

A rotated sorted array is a sorted array that has been shifted (rotated) at some pivot point. For example, `[4, 5, 6, 7, 0, 1, 2]` is a rotation of `[0, 1, 2, 4, 5, 6, 7]`. The problem is to search for a target in such an array in $O(\log n)$ time.

This is a classic variation of binary search that tests understanding of the binary search invariant.

2. Intuition

Simple explanation: A rotated sorted array is like a sorted array that has been cut at some point and the two halves were swapped. At any point, at least one half of the array is fully sorted. We can use this property to decide which half to search.

Analogy: Imagine a sorted deck of cards that you cut at some point. You swap the top and bottom halves. The deck is still "sorted" in the sense that each half is internally sorted, but the order is broken at the cut point. If you pick any card, you can tell which half it belongs to by checking the endpoints.

Step-by-step reasoning:

1. Find the midpoint.
2. Check if the left half (`arr[low]` to `arr[mid]`) is sorted: `arr[low] <= arr[mid]`.
3. If so, check if the target lies in this sorted left half (between `arr[low]` and `arr[mid]`).
 - If yes, search the left half.
 - If no, search the right half.
4. Otherwise, the right half (`arr[mid]` to `arr[high]`) must be sorted.
 - Check if the target lies in the sorted right half.
 - If yes, search the right half.
 - If no, search the left half.
5. Repeat until found or search space is empty.

Why it works: In a rotated sorted array, for any midpoint, at least one of the two halves is completely sorted. We can check which half is sorted by comparing `arr[low]` with `arr[mid]`. If the target is within the sorted half's

range, we search that half. Otherwise, we search the other half. This preserves the binary search property.

3. When to Use It

- **Rotated sorted array with distinct elements** — standard version.
- **Rotated sorted array with duplicates** — more complex version (worst case $O(n)$).
- **Finding minimum in rotated array** — related problem.
- **Finding the rotation count** — number of times the array was rotated.
- **Search in a rotated array with unknown pivot.**

Common trigger phrases: "rotated sorted array", "sorted and rotated", "circularly sorted", "shifted array", "pivot", "rotation", "find in rotated".

4. When Not to Use It

- **Array is not rotated** — just use standard binary search.
- **Array is not sorted** — this algorithm only works if the array is a rotation of a sorted array.
- **Array has many duplicates** — the algorithm degrades to $O(n)$ in the worst case (e.g., `[1, 1, 1, 1, 1, 1, 1]`).
- **Single search on small array** — linear search may be simpler and fast enough.
- **When you can identify the pivot first** — sometimes finding the pivot and then binary searching is simpler.

5. Core Concepts

Rotation: A sorted array `[a0, a1, ..., an-1]` rotated by k positions becomes `[ak, ak+1, ..., an-1, a0, a1, ..., ak-1]`.

Pivot: The point where the rotation happens (the smallest element in the array). In `[4, 5, 6, 7, 0, 1, 2]`, the pivot is at index 4 (value 0).

Sorted half property: In a rotated array, for any indices `low`, `mid`, `high`, at least one of the subarrays `[low..mid]` or `[mid..high]` is fully sorted.

Checking sortedness: If `arr[low] <= arr[mid]`, then the left half is sorted. Otherwise, the right half is sorted.

Duplicate handling: When `arr[low] == arr[mid] == arr[high]`, we cannot determine which half is sorted. We must shrink the search space by incrementing `low` and decrementing `high`.

6. Step-by-Step Algorithm

Without duplicates:

```
Input:  rotated sorted arr[0..n-1], target
Output: index of target, or -1
```

```
1. low = 0, high = n - 1
2. WHILE low <= high:
3.     mid = low + (high - low) / 2
```

```

4.     IF arr[mid] == target: RETURN mid
5.
6.     // Left half is sorted
7.     IF arr[low] <= arr[mid]:
8.         IF arr[low] <= target < arr[mid]:
9.             high = mid - 1    // target in left half
10.        ELSE:
11.            low = mid + 1     // target in right half
12.    // Right half is sorted
13.    ELSE:
14.        IF arr[mid] < target <= arr[high]:
15.            low = mid + 1     // target in right half
16.        ELSE:
17.            high = mid - 1    // target in left half
18.    RETURN -1

```

With duplicates (additional step when $\text{arr}[\text{low}] == \text{arr}[\text{mid}] == \text{arr}[\text{high}]$):

```

When arr[low] == arr[mid] == arr[high]:
    low++, high--    // shrink search space

```

7. Dry Run

Input: arr = [4, 5, 6, 7, 0, 1, 2], target = 0

Step	low	high	mid	arr[mid]	arr[low] <= arr[mid]?	Target in range?	Action
1	0	6	3	7	$4 \leq 7$ ✓ (left sorted)	$4 \leq 0 < 7$? No	low = 4
2	4	6	5	1	$0 \leq 1$ ✓ (left sorted)	$0 \leq 0 < 1$? Yes	high = 4
3	4	4	4	0	arr[4] == 0	Found	Return 4

Result: Index 4

Input: arr = [4, 5, 6, 7, 0, 1, 2], target = 3

Step	low	high	mid	arr[mid]	arr[low] <= arr[mid]?	Target in range?	Action
1	0	6	3	7	$4 \leq 7$ ✓ (left sorted)	$4 \leq 3 < 7$? No	low = 4
2	4	6	5	1	$0 \leq 1$ ✓ (left sorted)	$0 \leq 3 < 1$? No	low = 6
3	6	6	6	2	arr[6] == 3? No, $2 < 3$	$2 \leq 3 \leq 2$? No	high = 5
4	6	5	-	-	low > high	-	Return -1

Result: -1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Search in rotated sorted array (no duplicates)
int searchRotated(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) return mid;

        // Left half is sorted
        if (arr[low] <= arr[mid]) {
            if (arr[low] <= target && target < arr[mid]) {
                high = mid - 1; // Target in left sorted half
            } else {
                low = mid + 1; // Target in right half
            }
        }
        // Right half is sorted
        else {
            if (arr[mid] < target && target <= arr[high]) {
                low = mid + 1; // Target in right sorted half
            } else {
                high = mid - 1; // Target in left half
            }
        }
    }
    return -1;
}

// Search in rotated sorted array (with duplicates possible)
bool searchRotatedDuplicates(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) return true;

        // When we cannot determine which half is sorted
        if (arr[low] == arr[mid] && arr[mid] == arr[high]) {
            low++;
            high--;
        }
        // Left half is sorted
        else if (arr[low] <= arr[mid]) {
            if (arr[low] <= target && target < arr[mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
    }
}
```

```

        }
    }
    // Right half is sorted
    else {
        if (arr[mid] < target && target <= arr[high]) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
}
return false;
}

// Find minimum in rotated sorted array (no duplicates)
int findMinRotated(const vector<int>& arr) {
    int low = 0, high = (int)arr.size() - 1;

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] > arr[high]) {
            low = mid + 1; // Minimum is in right half
        } else {
            high = mid; // Minimum is in left half (including mid)
        }
    }
    return arr[low];
}

// Example usage
int main() {
    // No duplicates
    vector<int> arr = {4, 5, 6, 7, 0, 1, 2};
    cout << "Search for 0: " << searchRotated(arr, 0) << endl; // 4
    cout << "Search for 3: " << searchRotated(arr, 3) << endl; // -1
    cout << "Minimum: " << findMinRotated(arr) << endl; // 0

    // With duplicates
    vector<int> arr2 = {2, 5, 6, 0, 0, 1, 2};
    cout << "Search for 0 (dups): " << searchRotatedDuplicates(arr2, 0) << endl;
// 1

    return 0;
}

```

9. Python Implementation

```

from typing import List

# Search in rotated sorted array (no duplicates)

```



```
def search_rotated(arr: List[int], target: int) -> int:
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == target:
            return mid

        # Left half is sorted
        if arr[low] <= arr[mid]:
            if arr[low] <= target < arr[mid]:
                high = mid - 1
            else:
                low = mid + 1
        # Right half is sorted
        else:
            if arr[mid] < target <= arr[high]:
                low = mid + 1
            else:
                high = mid - 1

    return -1

# Search in rotated sorted array (with duplicates)
def search_rotated_duplicates(arr: List[int], target: int) -> bool:
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if arr[mid] == target:
            return True

        # Cannot determine which half is sorted
        if arr[low] == arr[mid] == arr[high]:
            low += 1
            high -= 1
        elif arr[low] <= arr[mid]:
            if arr[low] <= target < arr[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:
            if arr[mid] < target <= arr[high]:
                low = mid + 1
            else:
                high = mid - 1

    return False

# Find minimum in rotated sorted array
def find_min_rotated(arr: List[int]) -> int:
    low, high = 0, len(arr) - 1
```

```

while low < high:
    mid = low + (high - low) // 2

    if arr[mid] > arr[high]:
        low = mid + 1
    else:
        high = mid

return arr[low]

# Example usage
if __name__ == "__main__":
    arr = [4, 5, 6, 7, 0, 1, 2]
    print(f"Search for 0: {search_rotated(arr, 0)}") # 4
    print(f"Search for 3: {search_rotated(arr, 3)}") # -1
    print(f"Minimum: {find_min_rotated(arr)}") # 0

    arr2 = [2, 5, 6, 0, 0, 1, 2]
    print(f"Search for 0 (dups): {search_rotated_duplicates(arr2, 0)}") # True

```

10. Code Explanation

Standard version (no duplicates):

- The core binary search loop with `low <= high`.
- Compute `mid` as usual.
- If `arr[mid] == target`, return `mid`.
- **Determine which half is sorted:** Check if `arr[low] <= arr[mid]`. If true, the left half `[low..mid]` is sorted.
- **If left half is sorted:**
 - Check if target lies in the range `[arr[low], arr[mid]]`.
 - If yes, narrow to `high = mid - 1`.
 - If no, set `low = mid + 1`.
- **If right half is sorted:**
 - Check if target lies in the range `(arr[mid], arr[high]]`.
 - If yes, set `low = mid + 1`.
 - If no, set `high = mid - 1`.

Duplicate version:

- When `arr[low] == arr[mid] == arr[high]`, we cannot determine which half is sorted.
- Solution: increment `low` and decrement `high` to shrink the search space.
- This can degrade to $O(n)$ in the worst case (e.g., all elements equal).

Find minimum:

- Uses a different approach: compare `arr[mid]` with `arr[high]`.
- If `arr[mid] > arr[high]`: minimum is in the right half (`low = mid + 1`).
- Otherwise: minimum is in the left half (including `mid`) (`high = mid`).

- Loop continues while `low < high`. At the end, `low == high` points to the minimum.

11. Complexity Analysis

Variant	Time	Space
No duplicates (standard)	$O(\log n)$	$O(1)$
With duplicates (best case)	$O(\log n)$	$O(1)$
With duplicates (worst case)	$O(n)$	$O(1)$
Find minimum (no duplicates)	$O(\log n)$	$O(1)$
Find minimum (with duplicates)	$O(n)$ worst	$O(1)$

- **Best case:** Target is at mid, or array is not rotated.
- **Worst case:** With duplicates all equal, we may shrink one element at a time $\rightarrow O(n)$.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Search in rotated array	"Rotated sorted array", "search"	Check sorted half, decide direction	LeetCode 33
Search with duplicates	"Rotated" + "duplicates"	Add <code>arr[low] == arr[mid] == arr[high]</code> check	LeetCode 81
Find minimum	"Find minimum in rotated sorted array"	Compare mid with high	LeetCode 153
Find rotation count	"Find how many times array is rotated"	Find index of minimum element	GFG Rotation Count
Search in rotated array II	"Rotated" + "duplicates"	Same as pattern 2	LeetCode 81

13. Common Mistakes

- **Wrong sortedness check:** Using `arr[low] < arr[mid]` instead of `<=`. With 2 elements, `low=0`, `mid=0`, the condition `arr[0] < arr[0]` is false, so we incorrectly treat right half as sorted.
- **Incorrect range check for target:** The condition `arr[low] <= target < arr[mid]` uses `<=` on the left and `<` on the right. Getting this wrong causes off-by-one errors.
- **Not handling duplicates:** When `arr[low] == arr[mid] == arr[high]`, the algorithm fails without special handling.
- **Infinite loop in find minimum:** Using `low <= high` instead of `low < high` in the find-minimum variant.
- **Wrong mid update in find minimum:** Setting `high = mid - 1` instead of `high = mid` can skip the minimum.

14. Edge Cases

- Empty array: returns -1 (or false).
- Single element: `[5]`, search 5 → 0; search 3 → -1.
- Two elements: `[2, 1]`, search 2 → 0; search 1 → 1.
- No rotation (fully sorted): `[1, 2, 3, 4, 5]` — algorithm works.
- Rotated at first element: `[1, 2, 3, 4, 5]` — same as no rotation.
- Rotated at last element: `[2, 3, 4, 5, 1]` — works.
- All elements equal: `[1, 1, 1, 1]` — search returns true, but $O(n)$ time.
- Target is the smallest element (pivot).
- Target is the largest element.

15. Variations

Variation	What Changes	When Used	Importance
Search with duplicates	Handle <code>arr[low] == arr[mid] == arr[high]</code>	Real-world arrays with duplicates	High
Find minimum in rotated array	Return smallest element instead of searching	Finding pivot point	High
Find rotation count	Return index of minimum element	Know how many times rotated	Medium
Search in rotated array II	LeetCode 81 variant	Placement interview common	High
Search in a nearly sorted array	Element at most k positions away from sorted position	Different problem, not rotation	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Binary search	Base algorithm	If not rotated → standard binary search
Find pivot in rotated array	Subproblem for alternative approach	Can find pivot first, then binary search on the appropriate half
Lower/upper bound	Can be used after finding pivot	Find pivot, then <code>lower_bound</code> on the correct half

Alternative approach to search in rotated array: Find the pivot (minimum element index) using binary search. Then the array is sorted from pivot to $n-1$ and from 0 to $\text{pivot}-1$. Use binary search on both halves. This is more intuitive but requires two passes.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Find Minimum in Rotated Sorted Array	LeetCode 153	Find minimum in rotated array	Easy
Search in Rotated Sorted Array	LeetCode 33	Standard search in rotated array	Medium

Medium

Problem	Platform	Pattern	Difficulty
Search in Rotated Sorted Array II	LeetCode 81	Search with duplicates	Medium
Find Minimum in Rotated Sorted Array II	LeetCode 154	Find minimum with duplicates	Hard
Find Rotation Count	GFG	Count rotations	Medium

Hard

Problem	Platform	Pattern	Difficulty
Search in a Sorted Array of Unknown Size	LeetCode 702	Exponential + binary search	Medium
Recover Rotated Sorted Array	InterviewBit	Rotate back to sorted	Medium-Hard

18. Interview Explanation

"Search in a rotated sorted array is a variant of binary search. The key insight is that at any midpoint, at least one half of the array is fully sorted. We check if the left half is sorted by comparing `arr[low]` with `arr[mid]`. If the left half is sorted and the target lies within its range, we search the left half. Otherwise, we search the right half. If the left half is not sorted, the right half must be sorted. We apply the same logic. This maintains $O(\log n)$ time. For arrays with duplicates, when `arr[low]`, `arr[mid]`, and `arr[high]` are all equal, we cannot determine which half is sorted, so we shrink the search space by one from both ends, which can degrade to $O(n)$ in the worst case."

19. Revision Notes

- **Key insight:** At least one half is always sorted in a rotated sorted array.
- **Check sortedness:** `arr[low] <= arr[mid]` → left half sorted.
- **Range check (left sorted):** `arr[low] <= target < arr[mid]` → search left, else right.
- **Range check (right sorted):** `arr[mid] < target <= arr[high]` → search right, else left.
- **Duplicates:** When `arr[low] == arr[mid] == arr[high]`, do `low++`, `high--`.
- **Find minimum:** Compare `arr[mid]` with `arr[high]`. If `arr[mid] > arr[high]`, search right; else search left.
- **Complexity:** $O(\log n)$ without duplicates, up to $O(n)$ with duplicates.
- **Common trap:** Forgetting the `<=` in `arr[low] <= arr[mid]` for 2-element arrays.

20. Final Cheat Sheet

Aspect	Detail
When to use	Rotated sorted array, search for target, find minimum, find rotation count

Aspect	Detail
Main operation	Determine which half is sorted, check if target is in that half
Time	$O(\log n)$ no dups, $O(n)$ worst with dups
Space	$O(1)$
Key code	<pre>if (arr[low] <= arr[mid]) { if (arr[low] <= target < arr[mid]) high = mid-1; else low = mid+1; } else { ... }</pre>
Edge cases	Single element, two elements, all equal, no rotation, target at pivot

4. SEARCH IN 2D MATRIX

1. Overview

Searching in a 2D matrix involves finding a target value in an $m \times n$ matrix. Depending on the properties of the matrix (sorted rows, sorted columns, fully sorted), different approaches are used.

The two most common variants are:

1. **Matrix with sorted rows and sorted columns** (each row is sorted left to right, each column is sorted top to bottom — LeetCode 240).
2. **Matrix where each row is sorted and the first element of each row is greater than the last element of the previous row** (effectively a flattened sorted array — LeetCode 74).

2. Intuition

Simple explanation (fully sorted matrix): Imagine a 2D matrix that is sorted like one long array that has been broken into rows. You can treat it as a sorted 1D array and use binary search. The conversion from 1D index to 2D coordinates is `row = mid / n, col = mid % n`.

Simple explanation (row-wise + column-wise sorted): Imagine you are standing at the top-right corner of the matrix. The elements to your left are smaller, and the elements below are larger. By comparing the target with the current element, you can eliminate an entire row or column with each step.

Analogy (row-col sorted matrix): Think of a multiplication table. Numbers increase to the right and downward. If you are looking for a number, you can start at the top-right corner. If the target is smaller than the current number, move left (smaller numbers). If the target is larger, move down (larger numbers).

Step-by-step reasoning (fully sorted matrix):

1. Treat the matrix as a 1D sorted array of length $m \times n$.
2. Use standard binary search on this conceptual 1D array.
3. Convert the 1D mid index to 2D coordinates: `r = mid / n, c = mid % n`.
4. Compare `matrix[r][c]` with the target.
5. Adjust low/high as in standard binary search.

Step-by-step reasoning (row-col sorted matrix):

1. Start at the top-right corner: `row = 0, col = n - 1`.
2. While within bounds, compare `matrix[row][col]` with the target.
3. If equal, return true.
4. If the target is smaller, move left (`col--`) — because all elements in this column below are larger.
5. If the target is larger, move down (`row++`) — because all elements in this row to the left are smaller.
6. Repeat until found or out of bounds.

3. When to Use It

- **Fully sorted 2D matrix** (each row sorted, row starts $\geq$ previous row end) — use binary search on flattened index.
- **Row-wise and column-wise sorted matrix** — use the staircase (top-right) approach.
- **Each row is sorted individually** (but no column property) — binary search on each row, or binary search on the first column to find the correct row, then binary search on that row.
- **Matrix where rows are sorted and columns are sorted but not fully** — staircase approach works.

Common trigger phrases: "2D matrix", "sorted matrix", "search in matrix", "row and column sorted", "Young tableau", "staircase search".

4. When Not to Use It

- **Unsorted matrix** — no property to exploit. Use linear search $O(m \times n)$.
- **Very small matrix** (e.g., 2×2) — linear search is simpler.
- **Sparse matrix** — consider storing in a different data structure (hash set of coordinates).
- **When you need to search many times** — consider preprocessing (e.g., hash set of all elements).
- **Matrix with only rows sorted but not columns** — staircase approach may not work directly.

5. Core Concepts

Flattened index: For a matrix with m rows and n columns, the element at (r, c) corresponds to index $r * n + c$ in a flattened 1D array. The reverse: $r = \text{idx} / n, c = \text{idx} \% n$.

Staircase approach: Starting from the top-right (or bottom-left) corner, each step moves either left (decreasing column) or down (increasing row), forming a path that looks like a staircase.

Monotonicity in 2D: In a row-col sorted matrix, moving right increases the value, moving down increases the value. The matrix is monotonic in both directions.

Young tableau: A matrix where each row is sorted left-to-right and each column is sorted top-to-bottom. This is exactly the row-col sorted matrix property, and the search algorithm is the same as the standard Young tableau search.

6. Step-by-Step Algorithm

Fully sorted matrix (LeetCode 74):

Input: matrix[0..m-1][0..n-1] (fully sorted), target
Output: true/false

```
1. m = rows, n = cols
2. low = 0, high = m * n - 1
3. WHILE low <= high:
4.     mid = low + (high - low) / 2
5.     r = mid / n
6.     c = mid % n
7.     IF matrix[r][c] == target: RETURN true
8.     IF matrix[r][c] < target: low = mid + 1
9.     ELSE: high = mid - 1
10. RETURN false
```

Row-col sorted matrix (LeetCode 240):

Input: matrix[0..m-1][0..n-1] (row and column sorted), target
Output: true/false

```
1. row = 0, col = n - 1
2. WHILE row < m AND col >= 0:
3.     IF matrix[row][col] == target: RETURN true
4.     IF matrix[row][col] > target: col--      // move left
5.     ELSE: row++                          // move down
6. RETURN false
```

7. Dry Run

Fully sorted matrix: Matrix:

```
[1, 3, 5, 7]
[10, 11, 16, 20]
[23, 30, 34, 60]
```

Target: 3, m = 3, n = 4

Step	low	high	mid	r = mid/4	c = mid%4	matrix[r][c]	Comparison	Action
1	0	11	5	1	1	11	11 > 3	high = 4
2	0	4	2	0	2	5	5 > 3	high = 1
3	0	1	0	0	0	1	1 < 3	low = 1
4	1	1	1	0	1	3	3 == 3	Return true

Result: true

Row-col sorted matrix: Matrix:

```
[1, 4, 7, 11]
[2, 5, 8, 12]
[3, 6, 9, 16]
[10, 13, 14, 17]
```

Target: 5

Step	row	col	matrix[row][col]	Comparison	Action
1	0	3	11	11 > 5	col--
2	0	2	7	7 > 5	col--
3	0	1	4	4 < 5	row++
4	1	1	5	5 == 5	Return true

Result: true

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Variant 1: Fully sorted matrix (LeetCode 74)
// Each row is sorted, and first element of each row > last element of previous row
bool searchMatrixFullySorted(const vector<vector<int>>& matrix, int target) {
    int m = matrix.size();
    if (m == 0) return false;
    int n = matrix[0].size();
    if (n == 0) return false;

    int low = 0, high = m * n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        int r = mid / n;
        int c = mid % n;

        if (matrix[r][c] == target) return true;
        if (matrix[r][c] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return false;
}
```

```
// Variant 2: Row and column sorted matrix (LeetCode 240)
// Each row sorted left-to-right, each column sorted top-to-bottom
bool searchMatrixRowColSorted(const vector<vector<int>>& matrix, int target) {
    int m = matrix.size();
    if (m == 0) return false;
    int n = matrix[0].size();
    if (n == 0) return false;

    int row = 0, col = n - 1; // Start from top-right

    while (row < m && col >= 0) {
        if (matrix[row][col] == target) return true;
        if (matrix[row][col] > target) {
            col--; // Move left (smaller values)
        } else {
            row++; // Move down (larger values)
        }
    }
    return false;
}

// Variant 3: Binary search on each row (for when only rows are sorted)
bool searchMatrixRowsSorted(const vector<vector<int>>& matrix, int target) {
    for (const auto& row : matrix) {
        if (binary_search(row.begin(), row.end(), target)) {
            return true;
        }
    }
    return false;
}

// Variant 4: Binary search on first column, then on the row
bool searchMatrixTwoStep(const vector<vector<int>>& matrix, int target) {
    int m = matrix.size();
    if (m == 0) return false;
    int n = matrix[0].size();
    if (n == 0) return false;

    // Find the row where target could be
    int low = 0, high = m - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (matrix[mid][0] == target) return true;
        if (matrix[mid][0] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    // high is the last row where first element <= target
    int row = high;
    if (row < 0) return false;
```

```

    // Binary search on that row
    return binary_search(matrix[row].begin(), matrix[row].end(), target);
}

// Example usage
int main() {
    // Fully sorted (LeetCode 74)
    vector<vector<int>> mat1 = {
        {1, 3, 5, 7},
        {10, 11, 16, 20},
        {23, 30, 34, 60}
    };
    cout << "Fully sorted: " << searchMatrixFullySorted(mat1, 3) << endl;    // 1
    cout << "Fully sorted: " << searchMatrixFullySorted(mat1, 13) << endl; // 0

    // Row-col sorted (LeetCode 240)
    vector<vector<int>> mat2 = {
        {1, 4, 7, 11},
        {2, 5, 8, 12},
        {3, 6, 9, 16},
        {10, 13, 14, 17}
    };
    cout << "Row-col sorted: " << searchMatrixRowColSorted(mat2, 5) << endl;    //
1    cout << "Row-col sorted: " << searchMatrixRowColSorted(mat2, 20) << endl; //
0

    return 0;
}

```

9. Python Implementation

```

from typing import List
import bisect

# Variant 1: Fully sorted matrix (LeetCode 74)
def search_matrix_fully_sorted(matrix: List[List[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False

    m, n = len(matrix), len(matrix[0])
    low, high = 0, m * n - 1

    while low <= high:
        mid = low + (high - low) // 2
        r, c = mid // n, mid % n

        if matrix[r][c] == target:
            return True
        if matrix[r][c] < target:

```

```

        low = mid + 1
    else:
        high = mid - 1

    return False

# Variant 2: Row and column sorted matrix (LeetCode 240)
def search_matrix_rowcol_sorted(matrix: List[List[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False

    m, n = len(matrix), len(matrix[0])
    row, col = 0, n - 1 # Start from top-right

    while row < m and col >= 0:
        if matrix[row][col] == target:
            return True
        if matrix[row][col] > target:
            col -= 1
        else:
            row += 1

    return False

# Variant 3: Two-step approach (binary search on first column, then on row)
def search_matrix_two_step(matrix: List[List[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False

    m, n = len(matrix), len(matrix[0])

    # Find the row
    low, high = 0, m - 1
    while low <= high:
        mid = low + (high - low) // 2
        if matrix[mid][0] == target:
            return True
        if matrix[mid][0] < target:
            low = mid + 1
        else:
            high = mid - 1

    row = high
    if row < 0:
        return False

    # Binary search on the row
    idx = bisect.bisect_left(matrix[row], target)
    return idx < n and matrix[row][idx] == target

# Example usage
if __name__ == "__main__":
    # Fully sorted
    mat1 = [

```

```

    [1, 3, 5, 7],
    [10, 11, 16, 20],
    [23, 30, 34, 60]
]
print(f"Fully sorted: {search_matrix_fully_sorted(mat1, 3)}") # True
print(f"Fully sorted: {search_matrix_fully_sorted(mat1, 13)}") # False

# Row-col sorted
mat2 = [
    [1, 4, 7, 11],
    [2, 5, 8, 12],
    [3, 6, 9, 16],
    [10, 13, 14, 17]
]
print(f"Row-col sorted: {search_matrix_rowcol_sorted(mat2, 5)}") # True
print(f"Row-col sorted: {search_matrix_rowcol_sorted(mat2, 20)}") # False

```

10. Code Explanation

Fully sorted matrix:

- Treat the $m \times n$ matrix as a 1D array of length $m \times n$.
- `mid / n` gives the row, `mid % n` gives the column.
- Standard binary search on the conceptual 1D array.
- The division by `n` (number of columns) works because each row has exactly `n` elements.

Row-col sorted matrix (staircase):

- Start at `(0, n-1)` — top-right corner.
- The matrix is sorted right-to-left (decreasing) along the row, and top-to-bottom (increasing) along the column.
- At each step:
 - If `matrix[row][col] == target` → found.
 - If `matrix[row][col] > target` → move left (`col--`), because all elements below are even larger.
 - If `matrix[row][col] < target` → move down (`row++`), because all elements to the left are even smaller.
- Each step eliminates either a row or a column. Maximum steps: $m + n$.

Two-step approach:

- First, binary search on the first column to find the last row where `matrix[row][0] <= target`.
- Then, binary search on that row.
- Works for the fully sorted matrix variant (LeetCode 74).

11. Complexity Analysis

Variant	Time	Space
Fully sorted (binary search)	$O(\log(m \times n))$	$O(1)$
Row-col sorted (staircase)	$O(m + n)$	$O(1)$

Variant	Time	Space
Two-step (first column + row)	$O(\log m + \log n) = O(\log(m \times n))$	$O(1)$
Binary search on each row	$O(m \times \log n)$	$O(1)$

- **Fully sorted:** $O(\log(m \times n)) = O(\log m + \log n)$.
- **Staircase:** $O(m + n)$ in worst case (from top-right to bottom-left).
- **Two-step:** $O(\log m + \log n) = O(\log(m \times n))$.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Fully sorted 2D matrix	"Each row sorted, row starts > previous row ends"	Binary search on flattened array	LeetCode 74
Row-col sorted matrix	"Each row sorted, each column sorted"	Staircase from top-right or bottom-left	LeetCode 240
Rows sorted individually	"Each row sorted" (no column property)	Binary search each row	Various
Find kth smallest in sorted matrix	"Kth smallest", "sorted matrix"	Binary search on value range + count	LeetCode 378
Count negative numbers in sorted matrix	"Count negatives", "sorted matrix"	Staircase from bottom-left	LeetCode 1351

13. Common Mistakes

- **Forgetting that n (number of columns) is needed for mid-to-coordinate conversion:** $r = \text{mid} / n$, not mid / m .
- **Using mid / m instead of mid / n :** m = rows, n = cols. Flattened index = $r * n + c$, so $r = \text{mid} / n$.
- **Off-by-one in staircase:** Starting from $(0, n)$ instead of $(0, n-1)$ causes out-of-bounds.
- **Not handling empty matrix:** Check `matrix.size() == 0 || matrix[0].size() == 0`.
- **Assuming all rows have the same length** — not always true in some problems.
- **Using `int` for mid calculation** when $m * n$ can overflow — use `long long` if necessary.
- **Staircase from wrong corner:** Top-left and bottom-right do not work because both directions are increasing.

14. Edge Cases

- Empty matrix: `[]` or `[[] ]` — return false.
- Single row: `[[1, 2, 3]]` — both algorithms work.
- Single column: `[[1], [2], [3]]` — staircase works (top-right is just $(0, 0)$).
- Single element: `[[5]]` — search 5 $\rightarrow$ true; search 3 $\rightarrow$ false.
- Target smaller than all elements.
- Target larger than all elements.
- Target at first position $(0,0)$.

- Target at last position (m-1, n-1).
- Matrix with negative numbers.
- All elements equal to target.

15. Variations

Variation	What Changes	When Used	Importance
Fully sorted (LeetCode 74)	Row starts > previous row end	Matrix is effectively a 1D sorted array	High
Row-col sorted (LeetCode 240)	Only rows and columns individually sorted	More general, no full ordering	High
Kth smallest in sorted matrix	Find kth smallest, not search	More complex, uses binary search on value	High
Count negatives in sorted matrix	Count elements < 0	Follows from staircase pattern	Medium
Search in sorted matrix with rows sorted only	No column property	Weaker condition, $O(m \log n)$	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Binary search	Foundation for fully sorted variant	If matrix is fully sorted → flattened binary search
Staircase search	Specific to 2D sorted matrices	If row-col sorted → staircase
Divide and conquer	Can split matrix into quadrants	Alternative for row-col sorted, $O(n^{1.58})$

Decision rule:

- Fully sorted (each row's start > previous row's end) → binary search on flattened index.
- Row-wise and column-wise sorted → staircase (top-right or bottom-left).
- Only rows sorted → binary search on each row, or two-step approach.
- Not sorted → linear scan $O(m \times n)$.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Search a 2D Matrix	LeetCode 74	Fully sorted, binary search on flattened	Easy

Problem	Platform	Pattern	Difficulty
Count Negative Numbers in a Sorted Matrix	LeetCode 1351	Staircase from bottom-left	Easy

Medium

Problem	Platform	Pattern	Difficulty
Search a 2D Matrix II	LeetCode 240	Row-col sorted, staircase	Medium
Kth Smallest Element in a Sorted Matrix	LeetCode 378	Binary search on value range	Medium
Find a Peak Element II	LeetCode 1901	2D peak finding, binary search on rows	Medium

Hard

Problem	Platform	Pattern	Difficulty
Median of a Row Wise Sorted Matrix	GFG	Binary search on value range	Hard
Maximum Sum Rectangle in a 2D Matrix	GFG	Kadane's algorithm on 2D	Hard

18. Interview Explanation

"For searching in a 2D matrix, the approach depends on the matrix properties. If the matrix is fully sorted — each row is sorted and the first element of each row is greater than the last element of the previous row — I treat it as a 1D sorted array and use standard binary search. I convert the 1D mid index to 2D coordinates using `row = mid / n`, `col = mid % n`. This gives $O(\log(m \times n))$ time.

If the matrix is only row-wise and column-wise sorted (each row sorted, each column sorted), I use the staircase approach. I start from the top-right corner. If the current element is larger than the target, I move left. If it's smaller, I move down. Each step eliminates either a row or a column, giving $O(m + n)$ time. This is optimal for this type of matrix."

19. Revision Notes

- **Fully sorted matrix:** Binary search on flattened array. `r = mid / n`, `c = mid % n`.
- **Row-col sorted matrix:** Staircase from top-right. `if (mat[r][c] > target) c--; else r++;`.
- **Time:** $O(\log(m \times n))$ for fully sorted; $O(m + n)$ for staircase.
- **Space:** $O(1)$ for both.
- **Common trap:** Using `mid / m` instead of `mid / n` for coordinate conversion.
- **Common trap:** Starting staircase from top-left or bottom-right (doesn't work).
- **Starting positions that work:** Top-right $(0, n-1)$ or bottom-left $(m-1, 0)$.
- **Empty matrix:** Always check and return false.

20. Final Cheat Sheet

Aspect	Detail
When to use	Sorted 2D matrix, search for target
Fully sorted approach	Binary search on flattened index: <code>r = mid/n, c = mid%n</code>
Row-col sorted approach	Staircase from top-right: move left if larger, down if smaller
Time	$O(\log(m \times n))$ or $O(m + n)$
Space	$O(1)$
Key code (fully sorted)	<pre>while (low <= high) { mid = ...; r = mid/n; c = mid%n; ... }</pre>
Key code (staircase)	<pre>while (r < m && c >= 0) { if (mat[r][c] > target) c--; else r++; }</pre>
Edge cases	Empty matrix, single row/column, single element

5. LOWER BOUND / UPPER BOUND

1. Overview

Lower bound and upper bound are binary search variants that find insertion positions in a sorted array.

- **Lower bound:** Returns the first index where the value is $\geq$ **target** (i.e., the first position where the target could be inserted while maintaining sorted order).
- **Upper bound:** Returns the first index where the value is $>$ **target** (i.e., the first position after all occurrences of the target).

These are fundamental operations in C++ STL (`lower_bound`, `upper_bound`) and are essential for range queries, counting occurrences, and many algorithmic problems.

2. Intuition

Simple explanation: You have a sorted list of scores. You want to know where a new score of 85 would be placed to maintain sorted order. If there are already scores of 85, lower bound gives the position of the first 85, and upper bound gives the position after the last 85.

Analogy: Imagine a classroom with students seated in increasing height order. A new student arrives. Lower bound tells you where to seat them to maintain the order (if there are students of the same height, they sit at the first such position). Upper bound tells you to seat them after all students of the same height.

Step-by-step reasoning (lower bound):

1. We want to find the first index `i` such that `arr[i] >= target`.
2. Use binary search with `low = 0, high = n`.
3. At each step, compute `mid = low + (high - low) / 2`.
4. If `arr[mid] >= target`, we have a candidate. Move `high = mid` (search left for an earlier occurrence).
5. If `arr[mid] < target`, move `low = mid + 1` (search right).

6. When `low == high`, that is the answer.

Why it works: The predicate `arr[i] >= target` is monotonic — once it becomes true, it stays true for all larger indices. Binary search on a monotonic predicate always finds the first true value.

3. When to Use It

- **Finding the first position where a value $\geq$ target** (lower bound).
- **Finding the first position where a value $>$ target** (upper bound).
- **Counting occurrences of a value:** `upper_bound - lower_bound`.
- **Finding the closest element to a target** in a sorted array.
- **Range queries** in sorted arrays: "How many elements are between L and R?"
- **Insertion sort** — finding where to insert the next element.
- **Binary search on answer** — when the predicate is `arr[i] >= target`.

Common trigger phrases: "lower bound", "upper bound", "first element $\geq$ x", "first element $>$ x", "insert position", "count occurrences in sorted array", "range query", "floor and ceil".

4. When Not to Use It

- **Unsorted array** — lower/upper bound require sorted data.
- **Single search** — linear scan may be simpler.
- **When you need exact match only** — use standard binary search or `binary_search`.
- **When you need the last occurrence** — last occurrence is different from upper bound (upper bound gives the index after the last occurrence). Use last occurrence binary search or `upper_bound - 1`.
- **Non-comparable data types** — elements must support `<`, `>`, `>=`.

5. Core Concepts

Lower bound: `lower_bound(arr, target)` = smallest index `i` such that `arr[i] >= target`. If all elements are `<` target, returns `n` (`arr.size()`).

Upper bound: `upper_bound(arr, target)` = smallest index `i` such that `arr[i] > target`. If all elements are $\leq$ target, returns `n` (`arr.size()`).

Count of target: `upper_bound(arr, target) - lower_bound(arr, target)`.

Range of elements in [L, R]: `upper_bound(arr, R) - lower_bound(arr, L)`.

Distance from end: `n - upper_bound(arr, target)` gives the number of elements `>` target.

Monotonic predicate: A function `P(i)` that is false for all `i < k` and true for all `i  $\geq$  k`. Lower bound uses `P(i) = (arr[i] >= target)`.

High = n (not n-1): Unlike standard binary search, lower/upper bound use `high = n` because the answer can be `n` (when the target is greater than all elements).

6. Step-by-Step Algorithm

Lower bound:

Input: sorted arr[0..n-1], target
 Output: first index i such that arr[i] >= target

```

1. low = 0, high = n
2. WHILE low < high:
3.     mid = low + (high - low) / 2
4.     IF arr[mid] >= target:
5.         high = mid    // Potential answer, search left
6.     ELSE:
7.         low = mid + 1 // Search right
8. RETURN low // or high, both are same

```

Upper bound:

Input: sorted arr[0..n-1], target
 Output: first index i such that arr[i] > target

```

1. low = 0, high = n
2. WHILE low < high:
3.     mid = low + (high - low) / 2
4.     IF arr[mid] > target:
5.         high = mid    // Potential answer, search left
6.     ELSE:
7.         low = mid + 1 // Search right (arr[mid] <= target)
8. RETURN low

```

7. Dry Run

Lower bound: Input: arr = [1, 3, 3, 5, 7, 9], target = 3, n = 6

Step	low	high	mid	arr[mid]	arr[mid] >= 3?	Action
1	0	6	3	5	$5 \geq 3$ ✓	high = 3
2	0	3	1	3	$3 \geq 3$ ✓	high = 1
3	0	1	0	1	$1 \geq 3$ ✗	low = 1
4	1	1	-	-	low == high	Return 1

Result: Index 1 (first 3)

Upper bound: Input: arr = [1, 3, 3, 5, 7, 9], target = 3, n = 6

Step	low	high	mid	arr[mid]	arr[mid] > 3?	Action
1	0	6	3	5	$5 > 3$ ✓	high = 3
2	0	3	1	3	$3 > 3$ ✗	low = 2

Step	low	high	mid	arr[mid]	arr[mid] > 3?	Action
3	2	3	2	3	3 > 3 ✗	low = 3
4	3	3	-	-	low == high	Return 3

Result: Index 3 (first element > 3, which is 5 at index 3)

Count of 3: upper_bound(3) - lower_bound(3) = 3 - 1 = 2.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Custom lower bound
int lowerBound(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size(); // high = n, not n-1

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] >= target) {
            high = mid; // arr[mid] could be the answer, search left
        } else {
            low = mid + 1;
        }
    }
    return low; // first index where arr[i] >= target
}

// Custom upper bound
int upperBound(const vector<int>& arr, int target) {
    int low = 0, high = (int)arr.size();

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] > target) {
            high = mid; // arr[mid] could be the answer, search left
        } else {
            low = mid + 1; // arr[mid] <= target
        }
    }
    return low; // first index where arr[i] > target
}

// Count occurrences
int countOccurrences(const vector<int>& arr, int target) {
    int lb = lowerBound(arr, target);
    int ub = upperBound(arr, target);
    return ub - lb;
}
```

```

// Find floor (largest element <= target)
int floorValue(const vector<int>& arr, int target) {
    int lb = lowerBound(arr, target);
    if (lb < (int)arr.size() && arr[lb] == target) return arr[lb];
    if (lb == 0) return -1; // No floor
    return arr[lb - 1];
}

// Find ceil (smallest element >= target)
int ceilValue(const vector<int>& arr, int target) {
    int lb = lowerBound(arr, target);
    if (lb == (int)arr.size()) return -1; // No ceil
    return arr[lb];
}

// Example usage
int main() {
    vector<int> arr = {1, 3, 3, 5, 7, 9};

    cout << "Lower bound of 3: " << lowerBound(arr, 3) << endl; // 1
    cout << "Upper bound of 3: " << upperBound(arr, 3) << endl; // 3
    cout << "Count of 3: " << countOccurrences(arr, 3) << endl; // 2
    cout << "Floor of 4: " << floorValue(arr, 4) << endl; // 3
    cout << "Ceil of 4: " << ceilValue(arr, 4) << endl; // 5

    // Using STL
    cout << "STL lower_bound: " << (lower_bound(arr.begin(), arr.end(), 3) -
arr.begin()) << endl;
    cout << "STL upper_bound: " << (upper_bound(arr.begin(), arr.end(), 3) -
arr.begin()) << endl;

    return 0;
}

```

9. Python Implementation

```

from typing import List
import bisect

# Custom lower bound
def lower_bound(arr: List[int], target: int) -> int:
    low, high = 0, len(arr)

    while low < high:
        mid = low + (high - low) // 2

        if arr[mid] >= target:
            high = mid
        else:
            low = mid + 1

```

```

    return low

# Custom upper bound
def upper_bound(arr: List[int], target: int) -> int:
    low, high = 0, len(arr)

    while low < high:
        mid = low + (high - low) // 2

        if arr[mid] > target:
            high = mid
        else:
            low = mid + 1

    return low

# Count occurrences
def count_occurrences(arr: List[int], target: int) -> int:
    return upper_bound(arr, target) - lower_bound(arr, target)

# Floor
def floor_value(arr: List[int], target: int) -> int:
    lb = lower_bound(arr, target)
    if lb < len(arr) and arr[lb] == target:
        return arr[lb]
    if lb == 0:
        return -1
    return arr[lb - 1]

# Ceil
def ceil_value(arr: List[int], target: int) -> int:
    lb = lower_bound(arr, target)
    if lb == len(arr):
        return -1
    return arr[lb]

# Example usage
if __name__ == "__main__":
    arr = [1, 3, 3, 5, 7, 9]

    print(f"Lower bound of 3: {lower_bound(arr, 3)}") # 1
    print(f"Upper bound of 3: {upper_bound(arr, 3)}") # 3
    print(f"Count of 3: {count_occurrences(arr, 3)}") # 2
    print(f"Floor of 4: {floor_value(arr, 4)}") # 3
    print(f"Ceil of 4: {ceil_value(arr, 4)}") # 5

# Using bisect
print(f"bisect_left: {bisect.bisect_left(arr, 3)}") # 1
print(f"bisect_right: {bisect.bisect_right(arr, 3)}") # 3

```

10. Code Explanation

Lower bound implementation:

- `low = 0, high = n` — note that `high = n` (not `n-1`). This is because the answer could be `n` (when `target > all elements`).
- `while (low < high)` — NOT `<=`. We stop when `low == high`.
- `mid = low + (high - low) / 2`.
- If `arr[mid] >= target`: we have found a valid index. But there might be an earlier one. So we set `high = mid` (not `mid - 1`), because `mid` itself could be the answer.
- If `arr[mid] < target`: `mid` is too small. Set `low = mid + 1`.
- When the loop ends, `low == high == first index where arr[i] >= target`.

Upper bound implementation:

- Same structure, but the condition is `arr[mid] > target`.
- If `arr[mid] > target`: set `high = mid`.
- If `arr[mid] <= target`: set `low = mid + 1`.

Key difference from standard binary search:

- `high = n` (not `n-1`).
- `while (low < high)` (not `<=`).
- `high = mid` (not `mid - 1`) when the condition is met.
- Returns `low` (or `high`) instead of `-1` on failure.

11. Complexity Analysis

Operation	Time	Space
Lower bound	$O(\log n)$	$O(1)$
Upper bound	$O(\log n)$	$O(1)$
Count occurrences	$O(\log n)$	$O(1)$
Floor / Ceil	$O(\log n)$	$O(1)$

- **Time:** $O(\log n)$ for all operations.
- **Space:** $O(1)$ extra space.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Count occurrences	"Count of x in sorted array"	<code>upper_bound</code> - <code>lower_bound</code>	Count of element in sorted array
Range queries	"Count elements between L and R"	<code>upper_bound(R)</code> - <code>lower_bound(L)</code>	Count numbers in range
Floor/Ceil	"Largest $\leq x$ " or "Smallest $\geq x$ "	Use <code>lower_bound</code> , adjust	Floor and Ceil in sorted array

Pattern	How to Identify	Approach	Example Problems
Insert position	"Where should x be inserted?"	Lower bound (maintains sorted order)	Search Insert Position (LC 35)
Closest element	"Find closest to x"	Compare floor and ceil	Find closest element in sorted array
Kth missing number	"Kth missing positive number"	Binary search on missing count	Kth Missing Positive Number (LC 1539)

13. Common Mistakes

- **Using `high = n - 1` instead of `high = n`:** The answer can be `n` (when target > all elements). Using `n - 1` misses this case.
- **Using `while (low <= high)` instead of `while (low < high)`:** This causes infinite loops because `high = mid` does not decrease when `mid == high`.
- **Using `high = mid - 1` instead of `high = mid`:** When `arr[mid] >= target`, `mid` could be the answer. Setting `high = mid - 1` skips it.
- **Confusing lower and upper bound conditions:** Lower bound uses `>=`, upper bound uses `>`.
- **Not handling the return value of `n`:** When the target is greater than all elements, lower bound returns `n`. Accessing `arr[n]` is out of bounds.
- **Using `bisect_left` vs `bisect_right` in Python:** `bisect_left` = lower bound, `bisect_right` = upper bound.

14. Edge Cases

- Empty array: `[]`, lower bound of any target $\rightarrow 0$.
- Target smaller than all elements: `[5, 10, 15]`, lower bound of 3 $\rightarrow 0$.
- Target larger than all elements: `[5, 10, 15]`, lower bound of 20 $\rightarrow 3$.
- Target equal to first element: `[3, 5, 7]`, lower bound of 3 $\rightarrow 0$.
- Target equal to last element: `[3, 5, 7]`, lower bound of 7 $\rightarrow 2$.
- All elements equal: `[5, 5, 5, 5]`, lower bound of 5 $\rightarrow 0$, upper bound of 5 $\rightarrow 4$.
- Target not present: `[1, 3, 5, 7]`, lower bound of 4 $\rightarrow 2$ (index of 5).
- Single element: `[5]`, lower bound of 5 $\rightarrow 0$, lower bound of 10 $\rightarrow 1$.

15. Variations

Variation	What Changes	When Used	Importance
Lower bound with custom comparator	Predicate is not <code>>=</code> but some custom function	Binary search on answer, monotonic predicates	High
Upper bound with custom comparator	Condition is <code>></code> but on a custom property	Advanced problems	Medium
Lower bound on descending array	Reverse the condition	Arrays sorted in descending order	Medium

Variation	What Changes	When Used	Importance
Lower bound on a range of integers	Not an array but a continuous range	Binary search on answer	High
Last occurrence	Not exactly lower/upper bound, but related	Find last index where value = target	High

16. Related Algorithms

Algorithm	Connection	How to Choose
Binary search	Lower/upper bound are binary search variants	Need exact match → binary search; need range → lower/upper bound
Standard binary search	<code>binary_search</code> returns bool	Only need to know if element exists
Last occurrence search	Related to upper bound	<code>upper_bound - 1</code> gives last occurrence

STL equivalents:

- `lower_bound(arr.begin(), arr.end(), target)` → C++ lower bound.
- `upper_bound(arr.begin(), arr.end(), target)` → C++ upper bound.
- `bisect.bisect_left(arr, target)` → Python lower bound.
- `bisect.bisect_right(arr, target)` → Python upper bound.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Search Insert Position	LeetCode 35	Lower bound / insert position	Easy
First Bad Version	LeetCode 278	Lower bound on predicate	Easy

Medium

Problem	Platform	Pattern	Difficulty
Find First and Last Position of Element in Sorted Array	LeetCode 34	Lower bound + upper bound	Medium
Kth Missing Positive Number	LeetCode 1539	Lower bound on missing count	Medium
Count of Smaller Numbers After Self	LeetCode 315	Lower bound on sorted list (Fenwick)	Hard

Hard

Problem	Platform	Pattern	Difficulty
Median of Two Sorted Arrays	LeetCode 4	Binary search + lower bound concept	Hard
Range Sum Query - Mutable	LeetCode 307	Fenwick tree with lower bound	Hard

18. Interview Explanation

"Lower bound finds the first index in a sorted array where the element is $\geq$ target. Upper bound finds the first index where the element is $>$ target. Both use binary search in $O(\log n)$ time. The key difference from standard binary search is that we set `high = n` (not `n-1`) because the answer can be `n`. We use `while (low < high)` and `high = mid` when the condition is met, because `mid` itself could be the answer. Lower bound is useful for finding insert positions, counting occurrences (`upper_bound - lower_bound`), and range queries. In C++ STL, `lower_bound` and `upper_bound` are available. In Python, `bisect_left` and `bisect_right` serve the same purpose."

19. Revision Notes

- **Lower bound:** `arr[i] >= target`. First possible insert position.
- **Upper bound:** `arr[i] > target`. Position after last occurrence.
- **Count:** `upper_bound - lower_bound`.
- **Range [L, R]:** `upper_bound(R) - lower_bound(L)`.
- **Loop:** `while (low < high), high = arr.size()`.
- **Update:** `if (cond) high = mid; else low = mid + 1;`
- **Return:** `low` (or `high`).
- **STL:** `lower_bound()`, `upper_bound()` in C++.
- **Python:** `bisect_left()`, `bisect_right()`.
- **Common trap:** Setting `high = n - 1` instead of `n`.

20. Final Cheat Sheet

Aspect	Detail
When to use	Find insert position, count occurrences, range queries, floor/ceil
Lower bound	<code>while (low < high) { mid = ...; if (arr[mid] >= target) high = mid; else low = mid + 1; }</code>
Upper bound	Same but <code>if (arr[mid] > target) high = mid;</code>
Time	$O(\log n)$
Space	$O(1)$
Key difference from BS	<code>high = n, while (low < high), high = mid</code>
Edge cases	Empty array $\rightarrow 0$; target $>$ all $\rightarrow n$; target $<$ all $\rightarrow 0$

6. TERNARY SEARCH

1. Overview

Ternary search is a divide-and-conquer algorithm for finding the maximum or minimum of a **unimodal function** (a function that increases then decreases, or decreases then increases). It divides the search space into three equal parts and eliminates one of the outer thirds based on the function values at the two division points.

Unlike binary search (which works on sorted arrays), ternary search works on functions that are not monotonic but are unimodal.

2. Intuition

Simple explanation: You have a mountain range — it goes up, reaches a peak, then goes down. You want to find the peak. You pick two points on the mountain. If the left point is higher, the peak must be to the right of the left point. If the right point is higher, the peak must be to the left of the right point. By comparing two points, you eliminate one-third of the search space.

Analogy: Finding the highest point of a hill by taking two points on the slope. If the point on the left is lower than the point on the right, you know the peak is ahead (to the right). If the left is higher, the peak is behind (to the left). You can then discard the part that cannot contain the peak.

Step-by-step reasoning:

1. Take the current range $[l, r]$.
2. Divide it into three equal parts by picking two midpoints: $m1 = l + (r - l) / 3$, $m2 = r - (r - l) / 3$.
3. Evaluate $f(m1)$ and $f(m2)$.
4. **For finding maximum** (increasing then decreasing):
 - If $f(m1) < f(m2)$: the maximum is in $[m1, r]$ (discard left third).
 - If $f(m1) > f(m2)$: the maximum is in $[l, m2]$ (discard right third).
 - If equal: the maximum is in $[m1, m2]$.
5. Repeat until the range is small enough.

Why it works: A unimodal function has a single peak (or valley). By comparing the function values at two points, we can determine which side of the peak we are on and eliminate the part that cannot contain the extremum.

3. When to Use It

- **Unimodal functions** — functions that are strictly increasing then strictly decreasing (or vice versa).
- **Finding the maximum of a convex function** (more precisely, concave → unimodal).
- **Finding the minimum of a convex function** (more precisely, convex → unimodal).
- **Peak finding in mountain arrays** — where the array first increases then decreases.
- **Optimization problems** where the objective function is unimodal.
- **When the derivative is hard to compute** — ternary search is derivative-free.

Common trigger phrases: "unimodal", "mountain array", "peak", "convex function", "concave function", "find maximum of function", "bitonic array", "first increasing then decreasing".

4. When Not to Use It

- **Monotonic functions** — use binary search ($O(\log_2 n)$ vs $O(\log_3 n)$, binary is faster).
- **Non-unimodal functions** — functions with multiple peaks/valleys. Ternary search will fail.
- **Discrete functions with plateaus** — if the function has flat regions, ternary search may not work correctly.
- **When you need to find a specific value** — use binary search.
- **When the function is expensive to compute** — ternary search does 2 evaluations per iteration. Binary search does 1.
- **For integer arrays that are just sorted** — use binary search.

5. Core Concepts

Unimodal function: A function that is monotonic increasing up to a point (the maximum), then monotonic decreasing. The opposite (decreasing then increasing) is also unimodal (for finding minimum).

Two midpoints: Ternary search divides the range into three parts using $m1 = l + (r - l) / 3$ and $m2 = r - (r - l) / 3$.

Comparison logic:

- For finding maximum: $f(m1) < f(m2) \rightarrow$ peak is right of $m1$; $f(m1) > f(m2) \rightarrow$ peak is left of $m2$.
- For finding minimum: $f(m1) < f(m2) \rightarrow$ peak is left of $m2$; $f(m1) > f(m2) \rightarrow$ peak is right of $m1$.

Discrete ternary search: For integer arrays, the range is reduced until $r - l < 3$, then a linear scan finds the answer.

Continuous ternary search: For real-valued functions, iterate until $r - l < \text{epsilon}$ (a small precision value).

6. Step-by-Step Algorithm

Ternary search for maximum in a unimodal array:

```

Input:  arr[0..n-1] (unimodal: increasing then decreasing)
Output: index of maximum element

1. low = 0, high = n - 1
2. WHILE high - low >= 3:
3.     m1 = low + (high - low) / 3
4.     m2 = high - (high - low) / 3
5.     IF arr[m1] < arr[m2]:
6.         low = m1 + 1    // Max is in [m1+1, high]
7.     ELSE:
8.         high = m2 - 1   // Max is in [low, m2-1]
9. // Linear scan for the answer in the remaining small range
10. ans = low
11. FOR i = low TO high:
12.     IF arr[i] > arr[ans]:
13.         ans = i
14. RETURN ans

```

Ternary search for maximum of a continuous function:

Input: function f , range $[l, r]$, precision ϵ
 Output: x where $f(x)$ is maximum

```

1. WHILE  $r - l > \epsilon$ :
2.      $m1 = l + (r - l) / 3$ 
3.      $m2 = r - (r - l) / 3$ 
4.     IF  $f(m1) < f(m2)$ :
5.          $l = m1$ 
6.     ELSE:
7.          $r = m2$ 
8. RETURN  $l$  // or  $(l + r) / 2$ 

```

7. Dry Run

Discrete array: $\text{arr} = [1, 3, 7, 9, 8, 5, 2]$, $n = 7$

Step	low	high	m1	m2	arr[m1]	arr[m2]	arr[m1] < arr[m2]?	Action
1	0	6	2	4	7	8	$7 < 8$ ✓	low = 3
2	3	6	4	5	8	5	$8 < 5$ ✗	high = 4
3	3	4	-	-	range < 3 linear scan over [3,4]			

Linear scan: $\text{arr}[3] = 9$, $\text{arr}[4] = 8 \rightarrow$ max at index 3.

Result: Index 3 (value 9)

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Ternary search on discrete array (unimodal: increasing then decreasing)
// Find index of maximum element
int ternarySearchMax(const vector<int>& arr) {
    int n = arr.size();
    if (n == 0) return -1;
    if (n == 1) return 0;

    int low = 0, high = n - 1;

    while (high - low >= 3) {
        int m1 = low + (high - low) / 3;
        int m2 = high - (high - low) / 3;

        if (arr[m1] < arr[m2]) {
            low = m1 + 1; // Max is to the right of m1
        }
    }
}

```

```

        } else {
            high = m2 - 1; // Max is to the left of m2
        }
    }

    // Linear scan the remaining range
    int ans = low;
    for (int i = low; i <= high; i++) {
        if (arr[i] > arr[ans]) {
            ans = i;
        }
    }
    return ans;
}

// Ternary search on continuous function (find maximum)
// For finding minimum, reverse the comparison
double ternarySearchContinuous(double (*f)(double), double l, double r, double eps
= 1e-9) {
    while (r - l > eps) {
        double m1 = l + (r - l) / 3.0;
        double m2 = r - (r - l) / 3.0;

        if (f(m1) < f(m2)) {
            l = m1; // Maximum is to the right
        } else {
            r = m2; // Maximum is to the left
        }
    }
    return (l + r) / 2.0;
}

// Example: find minimum of f(x) = (x - 3)^2 + 5 (convex, minimum at x = 3)
double sampleFunction(double x) {
    return (x - 3) * (x - 3) + 5;
}

// For finding minimum, we use the opposite comparison
double ternarySearchMinContinuous(double (*f)(double), double l, double r, double
eps = 1e-9) {
    while (r - l > eps) {
        double m1 = l + (r - l) / 3.0;
        double m2 = r - (r - l) / 3.0;

        if (f(m1) < f(m2)) {
            r = m2; // Minimum is to the left
        } else {
            l = m1; // Minimum is to the right
        }
    }
    return (l + r) / 2.0;
}

// Example usage

```

```

int main() {
    // Discrete array
    vector<int> arr = {1, 3, 7, 9, 8, 5, 2};
    int idx = ternarySearchMax(arr);
    cout << "Maximum at index: " << idx << " (value: " << arr[idx] << ")" << endl;
    // Output: 3 (value: 9)

    // Continuous function
    double minX = ternarySearchMinContinuous(sampleFunction, -10, 10);
    cout << "Minimum of f(x) = (x-3)^2 + 5 at x = " << minX << endl;
    cout << "Minimum value: " << sampleFunction(minX) << endl;
    // Output: ~3, value: ~5

    return 0;
}

```

9. Python Implementation

```

from typing import List, Callable

# Ternary search on discrete array (maximum)
def ternary_search_max(arr: List[int]) -> int:
    n = len(arr)
    if n == 0:
        return -1
    if n == 1:
        return 0

    low, high = 0, n - 1

    while high - low >= 3:
        m1 = low + (high - low) // 3
        m2 = high - (high - low) // 3

        if arr[m1] < arr[m2]:
            low = m1 + 1
        else:
            high = m2 - 1

    # Linear scan remaining
    ans = low
    for i in range(low, high + 1):
        if arr[i] > arr[ans]:
            ans = i
    return ans

# Ternary search on continuous function (maximum)
def ternary_search_max_continuous(f: Callable[[float], float], l: float, r: float,
    eps: float = 1e-9) -> float:
    while r - l > eps:
        m1 = l + (r - l) / 3.0

```

```

        m2 = r - (r - l) / 3.0

        if f(m1) < f(m2):
            l = m1
        else:
            r = m2

    return (l + r) / 2.0

# Ternary search on continuous function (minimum)
def ternary_search_min_continuous(f: Callable[[float], float], l: float, r: float,
    eps: float = 1e-9) -> float:
    while r - l > eps:
        m1 = l + (r - l) / 3.0
        m2 = r - (r - l) / 3.0

        if f(m1) < f(m2):
            r = m2
        else:
            l = m1

    return (l + r) / 2.0

# Example usage
if __name__ == "__main__":
    # Discrete array
    arr = [1, 3, 7, 9, 8, 5, 2]
    idx = ternary_search_max(arr)
    print(f"Maximum at index: {idx} (value: {arr[idx]})") # 3 (9)

    # Continuous function
    def f(x):
        return (x - 3) ** 2 + 5

    min_x = ternary_search_min_continuous(f, -10, 10)
    print(f"Minimum at x = {min_x:.6f}, value = {f(min_x):.6f}") # ~3, ~5

```

10. Code Explanation

Discrete array version:

- Handle empty and single-element arrays.
- `low = 0`, `high = n - 1`.
- `while (high - low >= 3)` — continue while the range is large enough.
- Compute `m1` and `m2` (one-third and two-thirds positions).
- If `arr[m1] < arr[m2]`: the maximum is in the right 2/3, so set `low = m1 + 1`.
- If `arr[m1] >= arr[m2]`: the maximum is in the left 2/3, so set `high = m2 - 1`.
- After the loop, do a linear scan over the remaining small range (≤ 3 elements).
- This linear scan is critical because the ternary search narrowing may not pinpoint the exact index.

Continuous version:

- Uses **eps** (epsilon) for precision instead of exact integer comparison.
- The comparison determines whether we keep the left or right 2/3.
- For maximum: $f(m1) < f(m2) \rightarrow$ maximum is right, set $l = m1$.
- For minimum: $f(m1) < f(m2) \rightarrow$ minimum is left, set $r = m2$.

11. Complexity Analysis

Variant	Time	Space
Discrete ternary search	$O(\log_3 n)$	$O(1)$
Continuous ternary search	$O(\log_3((r-l)/\epsilon))$	$O(1)$

- **Time:** Each iteration reduces the range to 2/3 of its previous size. Number of iterations = $\log_3(n)$.
- **Comparison:** Binary search is $O(\log_2 n)$, which is faster than $O(\log_3 n)$ for the same n . However, ternary search works on a different class of problems.
- **Space:** $O(1)$ extra space.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Peak in mountain array	"First increasing then decreasing"	Ternary search or binary search on derivative	Peak Index in Mountain Array (LC 852)
Maximum of unimodal function	"Maximize $f(x)$ " with unimodal property	Ternary search on continuous range	Various optimization problems
Minimum of convex function	"Minimize $f(x)$ " with convex property	Ternary search for minimum	Various optimization problems
Bitonic array maximum	"Bitonic" = increasing then decreasing	Ternary search	Find max in bitonic array

13. Common Mistakes

- **Using ternary search on a non-unimodal function:** The algorithm assumes exactly one peak/valley. Multiple peaks will give wrong results.
- **Wrong comparison direction:** For maximum vs minimum, the comparison is reversed.
- **Not handling the final linear scan:** For discrete arrays, the loop ends when $high - low < 3$, but the answer could be in that small range. The linear scan is necessary.
- **Using $high - low \geq 3$ incorrectly:** Starting with a range that is too small may skip the loop entirely.
- **Using integer division for continuous version:** $m1 = l + (r - l) / 3$ works for both, but be careful with integer types.
- **Confusing $m1$ and $m2$:** $m1$ is closer to l , $m2$ is closer to r .
- **Setting eps too large or too small:** Too large $\rightarrow$ inaccurate. Too small $\rightarrow$ infinite loop or too many iterations.

14. Edge Cases

- Empty array: `[]` → -1.
- Single element: `[5]` → 0.
- Two elements: `[1, 2]` → 1 (max is at index 1).
- Three elements: `[1, 3, 2]` → 1.
- All decreasing: `[5, 4, 3, 2, 1]` → 0 (first element is maximum — but this is not strictly unimodal in the increasing-then-decreasing sense).
- All increasing: `[1, 2, 3, 4, 5]` → 4 (last element).
- Plateau at peak: `[1, 2, 3, 3, 3, 2, 1]` — ternary search may not work correctly with equal values. The comparison `arr[m1] < arr[m2]` may not eliminate the correct half.
- Continuous function with no minimum (e.g., linear): ternary search will converge to one end of the range.

15. Variations

Variation	What Changes	When Used	Importance
Ternary search for minimum	Reverse comparison	Convex functions (decreasing then increasing)	High
Discrete ternary search	Works on integer arrays	Bitonic/mountain array problems	High
Golden-section search	Uses golden ratio instead of 1/3	More efficient for continuous functions	Low (CP)
Ternary search on 2D	Extends to 2D functions	Finding maximum of a 2D unimodal surface	Low (rare)

16. Related Algorithms

Algorithm	Connection	How to Choose
Binary search	Both divide search space; binary is for monotonic, ternary for unimodal	Monotonic → binary; unimodal → ternary
Peak finding	Finding maximum in local neighborhood	1D peak → ternary or binary on derivative
Binary search on derivative	Alternative to ternary search for finding max/min	Ternary is simpler, derivative-based is more precise
Newton's method	Finds roots of derivative, faster convergence	When derivative is computable, Newton is faster

Note: For the "peak in mountain array" problem (LeetCode 852), binary search on the derivative (checking `arr[mid] < arr[mid+1]`) is actually more common and efficient than ternary search.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Peak Index in a Mountain Array	LeetCode 852	Find peak in bitonic array	Easy
Find Maximum in Bitonic Array	GFG	Maximum in bitonic array	Easy

Medium

Problem	Platform	Pattern	Difficulty
Find Peak Element	LeetCode 162	Find peak (binary search on derivative)	Medium
Minimum in a Bitonic Array	GFG	Minimum in bitonic array	Medium

Hard

Problem	Platform	Pattern	Difficulty
K-th Minimum in a Multiplication Table	LeetCode 378	Binary search on answer (not ternary)	Hard
Race Car	LeetCode 818	BFS + DP (not ternary)	Hard

18. Interview Explanation

"Ternary search is used to find the maximum or minimum of a unimodal function — a function that increases then decreases, or vice versa. We divide the range into three equal parts using two midpoints, $m1$ and $m2$. By comparing $f(m1)$ and $f(m2)$, we can eliminate one-third of the search space. For a maximum, if $f(m1) < f(m2)$, the peak is to the right, so we discard the left third. Otherwise, we discard the right third. The time complexity is $O(\log_3 n)$, and it's useful for optimization problems where the objective function is unimodal but you don't have a derivative. For discrete arrays, after narrowing down to a small range, we do a linear scan to find the exact peak."

19. Revision Notes

- **Purpose:** Find max/min of unimodal (bitonic) function.
- **Unimodal:** Increasing then decreasing (or vice versa), single peak/valley.
- **Two midpoints:** $m1 = l + (r-l)/3$, $m2 = r - (r-l)/3$.
- **Comparison for max:** $f(m1) < f(m2) \rightarrow$ peak is right; $f(m1) > f(m2) \rightarrow$ peak is left.
- **Comparison for min:** Opposite.
- **Complexity:** $O(\log_3 n)$ time, $O(1)$ space.
- **Discrete version:** Loop until $high - low < 3$, then linear scan.
- **Continuous version:** Loop until $r - l < \epsilon$.
- **Common trap:** Wrong comparison direction, not handling final linear scan for discrete.
- **Alternative:** Binary search on derivative (comparing $arr[mid]$ with $arr[mid+1]$).

20. Final Cheat Sheet

Aspect	Detail
When to use	Unimodal/bitonic array, convex/concave optimization
Main operation	Compare $f(m1)$ and $f(m2)$, discard one third
Time	$O(\log_3 n)$
Space	$O(1)$
Key code (discrete max)	<pre>while (high - low >= 3) { m1 = low + (high-low)/3; m2 = high - (high-low)/3; if (arr[m1] < arr[m2]) low = m1+1; else high = m2-1; }</pre>
Key code (continuous min)	<pre>while (r - l > eps) { m1 = ...; m2 = ...; if (f(m1) < f(m2)) r = m2; else l = m1; }</pre>
Edge cases	Single element, two elements, plateau at peak, non-unimodal input

7. EXPONENTIAL SEARCH

1. Overview

Exponential search (also called galloping search or doubling search) is an algorithm for searching in sorted arrays. It works in two phases:

1. Find a range that contains the target by repeatedly doubling the index.
2. Perform binary search on that range.

It is particularly useful for searching in **unbounded** or **infinite** arrays where the size is not known in advance.

2. Intuition

Simple explanation: You are looking for a word in a dictionary that has no page numbers. You first find a rough range by jumping exponentially: check page 1, then 2, then 4, then 8, then 16... until you go past the target. Then you binary search between the last two checked positions.

Analogy: Imagine you are looking for a house on a long street without house numbers. You start at house 1. If it's not there, you jump to house 2. Then 4. Then 8. Then 16. Once you've passed the target house, you binary search between the last two checkpoints.

Step-by-step reasoning:

1. Start at index 0. If `arr[0] == target`, return 0.
2. Start with `i = 1`. While `i < n` and `arr[i] <= target`, double `i`.
3. This gives a range `[i/2, min(i, n-1)]` that contains the target (if it exists).
4. Perform binary search on this range.

Why it works: The doubling phase ensures we find a range that contains the target in $O(\log k)$ time, where k is the position of the target. The subsequent binary search on a range of size $O(k)$ also takes $O(\log k)$ time. The total is $O(\log k)$, which is better than $O(\log n)$ when the target is near the beginning.

3. When to Use It

- **Unbounded/infinite arrays** — when you don't know the array size (LeetCode 702: Search in a Sorted Array of Unknown Size).
- **Target near the beginning** — when the target is expected to be close to index 0, exponential search is faster than binary search.
- **Linked lists with sorted data** — when you have a sorted linked list, binary search is $O(n)$ due to lack of random access, but exponential search can be adapted.
- **Very large arrays** — when n is extremely large, and you want to limit the search range first.
- **When binary search would be too slow because the array is huge** — exponential search finds a small range first.

Common trigger phrases: "unknown size", "infinite array", "sorted array of unknown size", "unbounded array", "search in infinite", "sorted linked list", "target near start".

4. When Not to Use It

- **Small arrays** — the overhead of doubling is unnecessary. Binary search is simpler.
- **Target near the end** — exponential search will double many times before finding the range. Binary search is $O(\log n)$ in all cases.
- **When the array size is known and the target is not near the beginning** — binary search is simpler and equally fast.
- **Unsorted data** — exponential search requires sorted data.
- **When you need to search many times** — sort once, then binary search each time. Exponential search's advantage is minimal for multiple searches.

5. Core Concepts

Doubling phase: Start with $i = 1$ and keep doubling ($i *= 2$) while $i < n$ and $arr[i] \leq target$. This finds an upper bound for the range.

Range: The range to binary search is $[i/2, \min(i, n-1)]$. The lower bound is the previous i value (before the last doubling), and the upper bound is the current i (clamped to $n-1$).

$O(\log k)$ time: If the target is at position k , the doubling phase takes $O(\log k)$ steps (since we double until we pass k), and the binary search on the range of size k also takes $O(\log k)$. Total: $O(\log k)$.

Advantage over binary search: Binary search is $O(\log n)$ regardless of where the target is. Exponential search is $O(\log k)$, which is better when k is small.

6. Step-by-Step Algorithm

```
Input:  sorted arr[0..n-1], target
Output: index of target, or -1
```

Phase 1: Find range

1. IF $arr[0] == target$: RETURN 0
2. $i = 1$
3. WHILE $i < n$ AND $arr[i] \leq target$:

```

4.     i = i * 2
5. // Now target is in range [i/2, min(i, n-1)]
6. RETURN binarySearch(arr, target, i/2, min(i, n-1))

```

7. Dry Run

Input: arr = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20], n = 10, target = 10

Step	i	arr[i]	arr[i] <= 10?	Action
1	0	arr[0] = 2	2 == 10? No	Continue
2	1	4	4 ≤ 10 ✓	i = 2
3	2	6	6 ≤ 10 ✓	i = 4
4	4	10	10 ≤ 10 ✓	i = 8
5	8	18	18 ≤ 10 ✗	Stop

Range: $[i/2, \min(i, n-1)] = [4, \min(8, 9)] = [4, 8]$

Binary search on arr[4..8] = [10, 12, 14, 16, 18]:

Step	low	high	mid	arr[mid]	Comparison	Action
1	4	8	6	14	14 > 10	high = 5
2	4	5	4	10	10 == 10	Return 4

Result: Index 4

Input: arr = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20], target = 1

Step	i	Action
1	0	arr[0] = 2, 2 != 1
2	1	arr[1] = 4, 4 ≤ 1? No

Range: $[i/2, \min(i, n-1)] = [0, 0]$

Binary search on arr[0..0] = [2], target 1 → -1.

Result: -1

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Binary search helper
int binarySearchRange(const vector<int>& arr, int target, int low, int high) {

```

```

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

// Exponential search
int exponentialSearch(const vector<int>& arr, int target) {
    int n = arr.size();
    if (n == 0) return -1;

    // Check first element
    if (arr[0] == target) return 0;

    // Find range by exponential doubling
    int i = 1;
    while (i < n && arr[i] <= target) {
        i *= 2;
    }

    // Binary search on range [i/2, min(i, n-1)]
    int low = i / 2;
    int high = min(i, n - 1);

    return binarySearchRange(arr, target, low, high);
}

// Exponential search for unbounded array (using indexer function)
// Useful when array size is unknown
int exponentialSearchUnbounded(const function<int(int)>& getElement, int target) {
    // Check first element
    if (getElement(0) == target) return 0;

    // Find range
    int i = 1;
    while (true) {
        int val = getElement(i);
        if (val == target) return i;
        if (val > target || val == INT_MAX) break; // INT_MAX = out of bounds
        i *= 2;
    }

    // Binary search on [i/2, i]
    int low = i / 2;
    int high = i;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        int val = getElement(mid);
        if (val == target) return mid;
    }
}

```

```

        if (val == INT_MAX || val > target) high = mid - 1;
        else low = mid + 1;
    }
    return -1;
}

// Example usage
int main() {
    vector<int> arr = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};

    cout << "Search for 10: " << exponentialSearch(arr, 10) << endl;    // 4
    cout << "Search for 1: " << exponentialSearch(arr, 1) << endl;      // -1
    cout << "Search for 20: " << exponentialSearch(arr, 20) << endl;    // 9
    cout << "Search for 3: " << exponentialSearch(arr, 3) << endl;      // -1

    return 0;
}

```

9. Python Implementation

```

from typing import List, Callable

# Binary search helper
def binary_search_range(arr: List[int], target: int, low: int, high: int) -> int:
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Exponential search
def exponential_search(arr: List[int], target: int) -> int:
    n = len(arr)
    if n == 0:
        return -1

    if arr[0] == target:
        return 0

    i = 1
    while i < n and arr[i] <= target:
        i *= 2

    low = i // 2
    high = min(i, n - 1)

    return binary_search_range(arr, target, low, high)

```



```

# Exponential search for unbounded array
# get_element(i) returns the value at index i, or None if out of bounds
def exponential_search_unbounded(get_element: Callable[[int], int], target: int) -
> int:
    if get_element(0) == target:
        return 0

    i = 1
    while True:
        val = get_element(i)
        if val == target:
            return i
        if val is None or val > target:
            break
        i *= 2

    low, high = i // 2, i

    while low <= high:
        mid = low + (high - low) // 2
        val = get_element(mid)
        if val == target:
            return mid
        if val is None or val > target:
            high = mid - 1
        else:
            low = mid + 1

    return -1

# Example usage
if __name__ == "__main__":
    arr = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

    print(f"Search for 10: {exponential_search(arr, 10)}") # 4
    print(f"Search for 1: {exponential_search(arr, 1)}") # -1
    print(f"Search for 20: {exponential_search(arr, 20)}") # 9
    print(f"Search for 3: {exponential_search(arr, 3)}") # -1

```

10. Code Explanation

Standard exponential search:

- First check `arr[0]` — if it matches, return 0.
- Initialize `i = 1` and double it while `i < n` and `arr[i] <= target`.
- After the loop, `i` is the first index where `arr[i] > target` (or `i >= n`).
- The range to binary search is `[i/2, min(i, n-1)]`.
- `i/2` is the last index where `arr[i] <= target` was true.
- `min(i, n-1)` ensures we don't go out of bounds.
- Call standard binary search on this range.

Unbounded array version:

- Uses a function `getElement(i)` that returns the value at index `i`.
- A sentinel value (like `INT_MAX` or `None`) indicates out-of-bounds.
- The doubling phase stops when `getElement(i) > target` or returns the sentinel.
- Binary search is similarly adapted.

11. Complexity Analysis

Case	Time	Space
Target at index <code>k</code> (known <code>n</code>)	$O(\log k)$	$O(1)$
Worst case (target at end)	$O(\log n)$	$O(1)$
Best case (target at index 0)	$O(1)$	$O(1)$
Binary search (for comparison)	$O(\log n)$	$O(1)$

- **Doubling phase:** $O(\log k)$ — doubles until we pass the target.
- **Binary search phase:** $O(\log k)$ — binary search on range of size `k`.
- **Total:** $O(\log k)$.
- **Best case:** $O(1)$ when target is at index 0.
- **Worst case:** $O(\log n)$ when target is at the end (same as binary search).

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Unbounded array	"Unknown size", "infinite array"	Exponential search with sentinel	Search in Sorted Array of Unknown Size (LC 702)
Target near start	"First few elements", "early"	Exponential search for faster initial range	Various
Sorted linked list	"Sorted linked list search"	Exponential search on linked list (pointer jumping)	Various
Range finding	"Find range before binary search"	Exponential search as preprocessing	Always applicable

13. Common Mistakes

- **Forgetting to check `arr[0]`** before the doubling loop. The loop starts with `i = 1`, so `arr[0]` is never checked.
- **Off-by-one in range:** The range should be `[i/2, min(i, n-1)]`, not `[i/2, i]`.
- **Not handling the case when `i` exceeds `n`:** The `while` condition `i < n` prevents this, but inside the binary search, `high = min(i, n-1)` is needed.
- **Integer overflow in `i *= 2`:** For very large arrays, `i` may overflow. Use `if (i > n/2) i = n;` or use `long long`.
- **Using exponential search on small arrays:** The overhead is unnecessary.

- **Not handling empty array.**

14. Edge Cases

- Empty array: `[]` → -1.
- Single element: `[5]`, search 5 → 0; search 3 → -1.
- Target at index 0: returns 0 immediately.
- Target at index 1: doubling phase: `i=1, arr[1] <= target`? Check. If yes, double to `i=2`. Range: `[1, min(2, n-1)]`.
- Target at last index: doubles until `i >= n`, then binary search on `[i/2, n-1]`.
- Target not present and smaller than all: `arr[0]` check fails, `i=1, arr[1] <= target` is false. Range: `[0, 0]`. Binary search on `[0, 0]` returns -1.
- Target not present and larger than all: doubles until `i >= n`, binary search on whole array returns -1.
- Array with negative numbers.

15. Variations

Variation	What Changes	When Used	Importance
Unbounded array search	No size known, use sentinel for out-of-bounds	Unknown size (LeetCode 702)	High
Exponential search on linked list	Use pointer jumping instead of indexing	Sorted linked list	Medium
Galloping search	Same as exponential search, used in timsort	Timsort merge algorithm	Low (internal)
Fibonacci search	Uses Fibonacci numbers instead of powers of 2	Similar to exponential, different progression	Low

16. Related Algorithms

Algorithm	Connection	How to Choose
Binary search	Binary search is the second phase	Known size + target anywhere → binary search
Jump search	Both use block-based approach	Jump search uses fixed block size, exponential uses growing blocks
Interpolation search	Both are improvements over binary search	Uniformly distributed data → interpolation
Linear search	Base case for small ranges	Very small n → linear search

Comparison with binary search:

- Binary search: $O(\log n)$ always, regardless of target position.
- Exponential search: $O(\log k)$ where k is target position. Better when k is small.
- For uniformly distributed targets, binary search is generally preferred.

- For unbounded arrays, exponential search is the only practical option.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Search in a Sorted Array of Unknown Size	LeetCode 702	Exponential search on unbounded array	Medium
Binary Search	LeetCode 704	Standard binary search	Easy

Medium

Problem	Platform	Pattern	Difficulty
Find the Index of the Large Integer	LeetCode 1533	Exponential + binary search	Medium
Search in Rotated Sorted Array	LeetCode 33	Can combine with exponential range finding	Medium

Hard

Problem	Platform	Pattern	Difficulty
Median of Two Sorted Arrays	LeetCode 4	Binary search (not exponential)	Hard
Koko Eating Bananas	LeetCode 875	Binary search on answer	Medium

18. Interview Explanation

"Exponential search works in two phases. First, we find a range that contains the target by starting at index 1 and doubling the index until we find an element greater than the target or reach the end of the array. Then, we perform binary search on the range between the previous index and the current index. The time complexity is $O(\log k)$, where k is the position of the target. This is better than binary search's $O(\log n)$ when the target is near the beginning of the array. It's also the algorithm of choice for searching in unbounded or infinite arrays where the size is not known in advance. The key idea is to find a small range first, then binary search within that range."

19. Revision Notes

- **Two phases:** Range finding (doubling) + binary search.
- **Start:** Check index 0, then start with $i = 1$.
- **Doubling:** `while (i < n && arr[i] <= target) i *= 2;`
- **Range:** $[i/2, \min(i, n-1)]$.
- **Complexity:** $O(\log k)$ where k = target position.
- **Best case:** $O(1)$ (target at index 0).

- **Worst case:** $O(\log n)$ (same as binary search).
- **Space:** $O(1)$.
- **Best use case:** Unbounded arrays, target near beginning.
- **Common trap:** Forgetting to check `arr[0]`, integer overflow in `i *= 2`.

20. Final Cheat Sheet

Aspect	Detail
When to use	Unbounded arrays, target near start, range finding + binary search
Phase 1	<code>i = 1; while (i < n && arr[i] <= target) i *= 2;</code>
Phase 2	Binary search on <code>[i/2, min(i, n-1)]</code>
Time	$O(\log k)$
Space	$O(1)$
Key code	<code>if (arr[0] == target) return 0; ... while (i < n && arr[i] <= target) i *= 2;</code>
Edge cases	Empty array, single element, target at index 0, target not present

8. JUMP SEARCH

1. Overview

Jump search (also called block search) is a searching algorithm for sorted arrays. It works by dividing the array into blocks of a fixed size ($\sqrt{n}$) and checking the last element of each block. When a block containing the target is found, linear search is performed within that block.

It is a middle ground between linear search ($O(n)$) and binary search ($O(\log n)$), offering $O(\sqrt{n})$ time complexity.

2. Intuition

Simple explanation: You have a sorted list of names. Instead of checking every name (linear) or jumping to the middle repeatedly (binary), you jump ahead by a fixed step size. You check if the current name is still less than the target. Once you overshoot, you go back one step and do a linear search within that block.

Analogy: Reading a book with page numbers. You don't read every page. You flip through in chunks of 10 pages. If the target page number is between page 30 and 40, you check the last page of each chunk — 10, 20, 30, 40. When you overshoot at 40, you go back to 30 and read linearly from 30 to 40.

Step-by-step reasoning:

1. Determine the block size: `step =  $\sqrt{n}$` .
2. Jump ahead by `step` until you find a block where the last element $\geq$ target.
3. Go back to the start of that block.

4. Perform linear search within the block.

Why it works: The optimal block size is $\sqrt{n}$, which balances the number of jumps ($n/\text{step} = \sqrt{n}$) and the linear search within the block ($\text{step} = \sqrt{n}$). The total is $O(\sqrt{n} + \sqrt{n}) = O(\sqrt{n})$.

3. When to Use It

- **Sorted arrays** where binary search is too complex or you want simpler code.
- **When memory is limited** — binary search is $O(\log n)$ and jump search is $O(\sqrt{n})$, but jump search uses less code space.
- **When the data is stored on slow-access media** (e.g., tape drives) — jumping reduces seeks compared to binary search.
- **When you need a balance between linear and binary search** — $O(\sqrt{n})$ is better than $O(n)$ but worse than $O(\log n)$.
- **Educational purposes** — understanding the trade-off between preprocessing and search.

Common trigger phrases: "sorted array", "block search", " $\sqrt{n}$ ", "jump search", "skip search", "sorted but not too large".

4. When Not to Use It

- **Large arrays** — binary search ($O(\log n)$) is exponentially faster.
- **Unsorted arrays** — jump search requires sorted data.
- **When you need $O(\log n)$ time** — use binary search.
- **Very small arrays** ($n < 20$) — linear search is simpler and may be faster due to cache behavior.
- **When the target is near the beginning** — the jump phase may still go through many blocks before finding the right one.
- **Linked lists** — no random access to jump by **step**.

5. Core Concepts

Block size: The optimal block size is $\sqrt{n}$. This minimizes the total cost: $n/\text{step} + \text{step}$ is minimized when $\text{step} = \sqrt{n}$.

Jump phase: Starting from index 0, add **step** to the index until the element at that index is $\geq$ target (or the end is reached).

Linear search phase: Once the block is found, perform linear search from the previous jump point to the current jump point.

Previous block: The start of the block is $\text{prev} = \text{Math.min}(\text{prev} + \text{step}, n-1)$ — the last jump point before the current one.

Trade-off: Jump search trades off between jump count and linear scan length. The optimal $\sqrt{n}$ comes from minimizing $n/\text{step} + \text{step}$.

6. Step-by-Step Algorithm

Input: sorted arr[0..n-1], target
 Output: index of target, or -1

```

1. n = arr.size()
2. step = sqrt(n)
3. prev = 0
4.
5. // Jump phase: find the block
6. WHILE arr[min(step, n) - 1] < target:
7.     prev = step
8.     step += sqrt(n)
9.     IF prev >= n: RETURN -1
10.
11. // Linear search within the block
12. FOR i = prev TO min(step, n) - 1:
13.     IF arr[i] == target: RETURN i
14. RETURN -1

```

7. Dry Run

Input: arr = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], n = 16, target = 10

step = $\sqrt{16} = 4$

Phase	prev	step	Check Index	arr[index]	arr[index] < 10?	Action
Jump	0	4	3	3	3 < 10 ✓	prev = 4, step = 8
Jump	4	8	7	7	7 < 10 ✓	prev = 8, step = 12
Jump	8	12	11	11	11 < 10 ✗	Stop

Block found: arr[8..11] = [8, 9, 10, 11]

Linear search in [8, 12):

i	arr[i]	arr[i] == 10?	Action
8	8	No	Continue
9	9	No	Continue
10	10	Yes	Return 10

Result: Index 10

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

```

```
// Jump search
int jumpSearch(const vector<int>& arr, int target) {
    int n = arr.size();
    if (n == 0) return -1;

    // Find block size
    int step = sqrt(n);

    // Find the block where the target may be present
    int prev = 0;
    while (arr[min(step, n) - 1] < target) {
        prev = step;
        step += sqrt(n);
        if (prev >= n) return -1;
    }

    // Linear search within the block
    while (prev < min(step, n)) {
        if (arr[prev] == target) {
            return prev;
        }
        prev++;
    }

    return -1;
}

// Jump search with custom block size (for performance tuning)
int jumpSearchCustom(const vector<int>& arr, int target, int blockSize) {
    int n = arr.size();
    if (n == 0) return -1;

    int prev = 0;
    while (arr[min(prev + blockSize, n) - 1] < target) {
        prev += blockSize;
        if (prev >= n) return -1;
    }

    while (prev < min(prev + blockSize, n)) {
        if (arr[prev] == target) return prev;
        prev++;
    }
    return -1;
}

// Example usage
int main() {
    vector<int> arr = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15};

    cout << "Search for 10: " << jumpSearch(arr, 10) << endl;    // 10
    cout << "Search for 0: " << jumpSearch(arr, 0) << endl;      // 0
    cout << "Search for 15: " << jumpSearch(arr, 15) << endl;    // 15
    cout << "Search for 16: " << jumpSearch(arr, 16) << endl;    // -1
    cout << "Search for -1: " << jumpSearch(arr, -1) << endl;    // -1
}
```



```
    return 0;
}
```

9. Python Implementation

```
import math
from typing import List

# Jump search
def jump_search(arr: List[int], target: int) -> int:
    n = len(arr)
    if n == 0:
        return -1

    # Find block size
    step = int(math.sqrt(n))

    # Find the block where the target may be present
    prev = 0
    while arr[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))
        if prev >= n:
            return -1

    # Linear search within the block
    while prev < min(step, n):
        if arr[prev] == target:
            return prev
        prev += 1

    return -1

# Example usage
if __name__ == "__main__":
    arr = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

    print(f"Search for 10: {jump_search(arr, 10)}")    # 10
    print(f"Search for 0: {jump_search(arr, 0)}")     # 0
    print(f"Search for 15: {jump_search(arr, 15)}")   # 15
    print(f"Search for 16: {jump_search(arr, 16)}")   # -1
    print(f"Search for -1: {jump_search(arr, -1)}")   # -1
```

10. Code Explanation

Jump search implementation:

- Compute `step = sqrt(n)` — the optimal block size.
- `prev = 0` — the start of the current block.

- `while (arr[min(step, n) - 1] < target)` — check the last element of the current block.
 - If it's still less than the target, the target is further ahead. Move `prev = step, step += sqrt(n)`.
 - `min(step, n)` prevents out-of-bounds access.
 - If `prev >= n`, the target is not in the array.
- After finding the block, linear search from `prev` to `min(step, n)`.
- Return the index if found, otherwise -1.

Optimization: The block size does not need to be exactly $\sqrt{n}$. Any block size works; $\sqrt{n}$ is optimal for worst-case performance.

11. Complexity Analysis

Case	Time	Space
Best case	$O(1)$	$O(1)$
Worst case	$O(\sqrt{n})$	$O(1)$
Average case	$O(\sqrt{n})$	$O(1)$

- **Best case:** Target is at the first element of the first block.
- **Worst case:** Target is at the last element of the last block. Number of jumps: $n/\sqrt{n} = \sqrt{n}$. Linear search within block: $\sqrt{n}$. Total: $\sqrt{n} + \sqrt{n} = O(\sqrt{n})$.
- **Space:** $O(1)$ extra space.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Standard jump search	Sorted array, $O(\sqrt{n})$ search	Jump by $\sqrt{n}$, linear search within block	Search in sorted array
Block-based search	"Block size", "chunk"	Generalization of jump search	Various
Hybrid search	Combine with other methods	Jump search for range, binary search within block	Various

13. Common Mistakes

- **Off-by-one in block checking:** Checking `arr[step]` instead of `arr[step - 1]` for the last element of the block.
- **Not using `min(step, n)`:** This causes out-of-bounds access when `step` exceeds `n`.
- **Incorrect block size:** Using `step = n` (degenerates to linear), `step = 1` (degenerates to linear), or `step = n/2` (too few jumps, too much linear search).
- **Not handling empty array.**
- **Integer overflow in `step` calculation** for very large `n`.
- **Using `sqrt` with floating point:** `sqrt(n)` returns a double. Cast to int properly.

14. Edge Cases

- Empty array: `[]` → -1.
- Single element: `[5]`, search 5 → 0; search 3 → -1.
- Target at first element: found immediately in the linear search phase.
- Target at last element: jumps through all blocks, then linear search finds it.
- Target not present (smaller than all): the first block's last element is > target, linear search in the first block fails.
- Target not present (larger than all): jumps past the end, `prev >= n`, returns -1.
- Array size is a perfect square: `n = 16`, `step = 4` — works cleanly.
- Array size is not a perfect square: `n = 10`, `step = 3` — last block may be smaller.

15. Variations

Variation	What Changes	When Used	Importance
Jump search with binary search	Use binary search instead of linear within the block	Better performance: $O(\log \sqrt{n})$ instead of $O(\sqrt{n})$	Low
Jump search with custom block size	Block size is not $\sqrt{n}$	When access cost is known	Low
Block search on linked list	Jump by following pointers	Sorted linked list (rare)	Low

16. Related Algorithms

Algorithm	Connection	How to Choose
Linear search	Jump search is linear search with jumps	$O(n)$ → linear; $O(\sqrt{n})$ → jump; $O(\log n)$ → binary
Binary search	Both work on sorted arrays	Binary is faster ($O(\log n)$ vs $O(\sqrt{n})$)
Exponential search	Both use a "jump" approach	Exponential uses growing jumps, jump uses fixed jumps
Interpolation search	Both improve over binary search	Uniform distribution → interpolation

Comparison with binary search:

- Binary search: $O(\log n)$. Always better for large n .
- Jump search: $O(\sqrt{n})$. Simpler to implement.
- Why use jump search? Mostly educational. In practice, binary search is almost always preferred for sorted arrays.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Search a 2D Matrix	LeetCode 74	Binary search (not jump)	Easy
Binary Search	LeetCode 704	Binary search	Easy

Medium

Problem	Platform	Pattern	Difficulty
Search in Rotated Sorted Array	LeetCode 33	Rotated binary search	Medium
Find First and Last Position	LeetCode 34	Lower/upper bound	Medium

Hard

Problem	Platform	Pattern	Difficulty
Median of Two Sorted Arrays	LeetCode 4	Binary search	Hard
Kth Smallest Element in a Sorted Matrix	LeetCode 378	Binary search on value	Hard

18. Interview Explanation

"Jump search is a searching algorithm for sorted arrays. It divides the array into blocks of size $\sqrt{n}$ and checks the last element of each block. Once we find a block where the last element is $\geq$ target, we perform linear search within that block. The optimal block size is $\sqrt{n}$, which balances the number of jumps ($n/\sqrt{n} = \sqrt{n}$) and the linear search within the block ($\sqrt{n}$), giving $O(\sqrt{n})$ time complexity. It's a middle ground between linear search $O(n)$ and binary search $O(\log n)$. In practice, binary search is almost always preferred for sorted arrays, but jump search is a good illustration of the trade-off between jumping and scanning."

19. Revision Notes

- **Block size:** $\text{step} = \sqrt{n}$ (optimal).
- **Jump phase:** Check $\text{arr}[\text{step}-1]$, $\text{arr}[2*\text{step}-1]$, ... until $\geq$ target.
- **Linear phase:** Search within the block from prev to $\min(\text{step}, n)$.
- **Complexity:** $O(\sqrt{n})$ time, $O(1)$ space.
- **Best case:** $O(1)$ (target at start).
- **Worst case:** $O(\sqrt{n})$ (target at end).
- **Requires:** Sorted array, random access.
- **Common trap:** Forgetting $\min(\text{step}, n)$ causing out-of-bounds.
- **Comparison:** Binary search ($O(\log n)$) is always better for large n .

20. Final Cheat Sheet

Aspect	Detail
When to use	Sorted array, want $O(\sqrt{n})$ search, simpler than binary

Aspect	Detail
Block size	<code>step = sqrt(n)</code>
Jump	<code>while (arr[min(step, n)-1] < target) { prev = step; step += sqrt(n); }</code>
Linear scan	<code>for (i = prev; i < min(step, n); i++) if (arr[i] == target) return i;</code>
Time	$O(\sqrt{n})$
Space	$O(1)$
Key code	<pre>int step = sqrt(n); while (arr[min(step, n)-1] < target) { prev = step; step += sqrt(n); }</pre>
Edge cases	Empty array, single element, target at ends, target not present

9. INTERPOLATION SEARCH

1. Overview

Interpolation search is an improved variant of binary search for sorted arrays. Instead of always checking the middle element, it estimates the position of the target based on the value — like looking up a word in a dictionary by estimating its approximate position based on the first letter.

The probe position is calculated using the formula: `pos = low + (target - arr[low]) * (high - low) / (arr[high] - arr[low])`.

2. Intuition

Simple explanation: When you look up a word in a dictionary starting with 'Z', you don't open the book in the middle. You open it near the end. Interpolation search does the same — it uses the value of the target to estimate its position, assuming the data is uniformly distributed.

Analogy: Finding a person's age in a sorted list of ages 1-100. If you're looking for age 90, you wouldn't start at the middle (age 50). You'd jump to near the end, because 90 is close to 100. Interpolation search uses the target value to estimate the index.

Step-by-step reasoning:

1. Given a sorted array and a target, compute the probable position using linear interpolation.
2. If the element at the probed position matches the target, return it.
3. If the target is smaller, adjust the search range to the left of the probed position.
4. If the target is larger, adjust the search range to the right.
5. Repeat until the target is found or the range is empty.

Why it works: Interpolation search assumes the data is uniformly distributed. The formula estimates the position by assuming the values increase linearly with the index. For uniformly distributed data, this estimate is very accurate, often finding the target in $O(\log \log n)$ comparisons.

3. When to Use It

- **Sorted array with uniformly distributed values** — this is the ideal case.
- **Large arrays** — $O(\log \log n)$ is significantly faster than $O(\log n)$ for large n .
- **When the cost of comparison is high** — fewer comparisons needed.
- **When you know the data is uniformly distributed** — e.g., IDs, sequential numbers, ages.

Common trigger phrases: "uniformly distributed", "sorted array", "interpolation search", "probe search", "dictionary search", "estimate position".

4. When Not to Use It

- **Non-uniformly distributed data** — worst case is $O(n)$ (e.g., exponential data: [1, 2, 4, 8, 16, ...]).
- **Small arrays** — binary search is simpler and fast enough.
- **Unsorted arrays** — requires sorted data.
- **Non-numeric data** — the formula requires arithmetic on values (subtraction, multiplication).
- **When values are not distinct** — duplicates can cause issues (but not critically).
- **When overflow is a concern** — the formula can overflow for large values.

5. Core Concepts

Probe position formula: $pos = low + (target - arr[low]) * (high - low) / (arr[high] - arr[low])$.

This formula maps the target value to a position in the range [low, high] using linear interpolation. It assumes $arr[low]$ maps to `low` and $arr[high]$ maps to `high`.

Uniform distribution: The algorithm works best when the values are uniformly distributed across the range. For example, an array of 1000 integers from 0 to 999.

$O(\log \log n)$ time: For uniformly distributed data, the expected number of comparisons is $\log \log n$. This is because each probe divides the range proportionally, and the expected range size reduces exponentially.

Worst-case $O(n)$: For non-uniform distributions (e.g., exponential growth), the probe may be very inaccurate, and the algorithm degenerates to $O(n)$.

6. Step-by-Step Algorithm

Input: sorted $arr[0..n-1]$, target

Output: index of target, or -1

```

1. low = 0, high = n - 1
2. WHILE low <= high AND target >= arr[low] AND target <= arr[high]:
3.     IF low == high:
4.         IF arr[low] == target: RETURN low
5.         ELSE: RETURN -1
6.
7.     // Estimate position
8.     pos = low + (target - arr[low]) * (high - low) / (arr[high] - arr[low])
9.
10.    IF arr[pos] == target: RETURN pos
11.    IF arr[pos] < target: low = pos + 1

```

```
12.     ELSE: high = pos - 1
13. RETURN -1
```

7. Dry Run

Input: arr = [10, 12, 13, 16, 18, 19, 20, 21, 22, 23, 24, 33, 35, 42, 47], n = 15, target = 18

Step	low	high	arr[low]	arr[high]	Formula	pos	arr[pos]	Comparison	Action
1	0	14	10	47	$0 + (18-10) \cdot \frac{(14-0)}{(47-10)}$ $= 0 + 8 \cdot \frac{14}{37}$ $= 3$	3	16	$16 < 18$	low = 4
2	4	14	18	47	$4 + (18-18) \cdot \frac{(14-4)}{(47-18)}$ $= 4 + 0$	4	18	$18 == 18$	Return 4

Result: Index 4 (found in 2 steps)

Input: arr = [10, 12, 13, 16, 18, 19, 20, 21, 22, 23, 24, 33, 35, 42, 47], target = 25

Step	low	high	pos	arr[pos]	Comparison	Action
1	0	14	7	21	$21 < 25$	low = 8
2	8	14	10	24	$24 < 25$	low = 11
3	11	14	12	35	$35 > 25$	high = 11
4	11	11	11	33	$33 > 25$	high = 10
5	11	10	-	-	low > high	Return -1

Result: -1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Interpolation search
int interpolationSearch(const vector<int>& arr, int target) {
    int n = arr.size();
    int low = 0, high = n - 1;

    while (low <= high && target >= arr[low] && target <= arr[high]) {
        // If only one element remains
        if (low == high) {
            return (arr[low] == target) ? low : -1;
        }
    }
}
```

```

        // Estimate the position using interpolation formula
        // pos = low + ((target - arr[low]) * (high - low)) / (arr[high] -
arr[low])
        int pos = low + ((target - arr[low]) * (high - low)) / (arr[high] -
arr[low]);

        // Check if the estimated position is within bounds
        if (pos < low || pos > high) return -1;

        if (arr[pos] == target) return pos;
        if (arr[pos] < target) low = pos + 1;
        else high = pos - 1;
    }

    return -1;
}

// Example usage
int main() {
    // Uniformly distributed data (ideal for interpolation search)
    vector<int> arr = {10, 12, 13, 16, 18, 19, 20, 21, 22, 23, 24, 33, 35, 42,
47};

    cout << "Search for 18: " << interpolationSearch(arr, 18) << endl;    // 4
    cout << "Search for 10: " << interpolationSearch(arr, 10) << endl;    // 0
    cout << "Search for 47: " << interpolationSearch(arr, 47) << endl;    // 14
    cout << "Search for 25: " << interpolationSearch(arr, 25) << endl;    // -1
    cout << "Search for 5: " << interpolationSearch(arr, 5) << endl;      // -1
    cout << "Search for 50: " << interpolationSearch(arr, 50) << endl;    // -1

    return 0;
}

```

9. Python Implementation

```

from typing import List

# Interpolation search
def interpolation_search(arr: List[int], target: int) -> int:
    n = len(arr)
    low, high = 0, n - 1

    while low <= high and arr[low] <= target <= arr[high]:
        if low == high:
            return low if arr[low] == target else -1

        # Estimate position
        pos = low + ((target - arr[low]) * (high - low)) // (arr[high] - arr[low])

        if pos < low or pos > high:
            return -1

```



```

    if arr[pos] == target:
        return pos
    if arr[pos] < target:
        low = pos + 1
    else:
        high = pos - 1

    return -1

# Example usage
if __name__ == "__main__":
    arr = [10, 12, 13, 16, 18, 19, 20, 21, 22, 23, 24, 33, 35, 42, 47]

    print(f"Search for 18: {interpolation_search(arr, 18)}")    # 4
    print(f"Search for 10: {interpolation_search(arr, 10)}")    # 0
    print(f"Search for 47: {interpolation_search(arr, 47)}")    # 14
    print(f"Search for 25: {interpolation_search(arr, 25)}")    # -1
    print(f"Search for 5: {interpolation_search(arr, 5)}")      # -1
    print(f"Search for 50: {interpolation_search(arr, 50)}")    # -1

```

10. Code Explanation

Interpolation search:

- `low = 0, high = n - 1` — standard binary search range.
- `while (low <= high && target >= arr[low] && target <= arr[high])` — the second condition checks if the target is within the current range. If the target is outside the range of values, we can stop early.
- `if (low == high)` — only one element left. Check and return.
- `pos = low + ((target - arr[low]) * (high - low)) / (arr[high] - arr[low])` — the interpolation formula. This estimates the position of the target assuming linear distribution.
- `if (pos < low || pos > high)` — safety check. If the formula produces an out-of-bounds position (can happen with duplicates or edge cases), return -1.
- Three-way comparison: `arr[pos] == target` → return; `arr[pos] < target` → search right; `arr[pos] > target` → search left.

Safety considerations:

- The formula uses integer arithmetic. The multiplication `(target - arr[low]) * (high - low)` can overflow for large values. Using `long long` in C++ is recommended.
- `arr[high] - arr[low]` can be zero (all elements equal). This causes division by zero. The `target >= arr[low] && target <= arr[high]` condition with `target >= arr[low]` implies `target == arr[low]` when all values are equal, so we can handle this case separately.

11. Complexity Analysis

Distribution	Time	Space
Uniform (expected)	$O(\log \log n)$	$O(1)$

Distribution	Time	Space
Non-uniform (worst case)	$O(n)$	$O(1)$
Best case	$O(1)$	$O(1)$

- **Best case:** Target is found at the first probe.
- **Average case (uniform):** $O(\log \log n)$ — the range reduces exponentially.
- **Worst case:** $O(n)$ — for non-uniform data like exponential growth (e.g., [1, 2, 4, 8, 16, ...]).
- **Space:** $O(1)$ extra space.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Uniformly distributed sorted array	Values are evenly spread	Interpolation search	Search in sorted uniform array
Dictionary search	Strings sorted by word	Can use interpolation on the first character	Various
Sorted array with unknown distribution	Don't know distribution	Use binary search (safe $O(\log n)$)	Standard binary search
Interpolation + binary hybrid	Detect distribution dynamically	Switch to binary if interpolation is slow	Advanced implementations

13. Common Mistakes

- **Division by zero:** When `arr[high] == arr[low]`, the denominator is zero. The while condition `target >= arr[low] && target <= arr[high]` with `arr[low] == arr[high]` implies `target == arr[low]`, so we should handle this case.
- **Integer overflow:** `(target - arr[low]) * (high - low)` can overflow for large ints. Use `long long` in C++.
- **Not checking bounds:** `pos` may be out of bounds if the formula is inaccurate. Always check `pos >= low && pos <= high`.
- **Using on non-uniform data:** The algorithm degrades to $O(n)$ for non-uniform distributions.
- **Using on non-numeric data:** The formula requires arithmetic operations.
- **Not checking corner conditions:** The while loop condition should include `target >= arr[low] && target <= arr[high]` to exit early when the target is out of range.

14. Edge Cases

- Empty array: `[]` → -1.
- Single element: `[5]`, search 5 → 0; search 3 → -1.
- All elements equal: `[5, 5, 5, 5]` — the formula causes division by zero. The while condition `target >= arr[low] && target <= arr[high]` with all equal means `target == 5`. We should handle this.
- Target out of range: `[10, 20, 30]`, search 5 → the while condition `target >= arr[low]` fails, loop exits, returns -1.
- Target at first position: `arr[0] == target` — the formula should estimate `pos ≈ 0`.

- Target at last position: `arr[n-1] == target` — the formula should estimate $\text{pos} \approx n-1$.
- Non-uniform data: `[1, 2, 4, 8, 16, 32, 64, 128]`, search 64 — the probe may be very inaccurate.
- Duplicate values: may cause the probe to be slightly off, but the algorithm still works.

15. Variations

Variation	What Changes	When Used	Importance
Interpolation search with binary fallback	If probe is not accurate, switch to binary	Non-uniform data	Medium
Quadratic interpolation search	Uses quadratic formula instead of linear	More accurate for some distributions	Low
Fibonacci search	Uses Fibonacci numbers for probe positions	Similar to interpolation but division-free	Low

16. Related Algorithms

Algorithm	Connection	How to Choose
Binary search	Both search sorted arrays	Binary is safer ($O(\log n)$ always); interpolation is faster only for uniform data
Exponential search	Both use estimation	Exponential for unbounded; interpolation for uniform bounded
Jump search	Both use jumping	Jump uses fixed jumps; interpolation uses value-based jumps
Binary search tree	Different data structure	Array $\rightarrow$ interpolation; tree $\rightarrow$ BST

Key insight: Interpolation search is only beneficial when the data is uniformly distributed and the array is large. For most competitive programming and placement problems, binary search is the safer and more commonly used choice.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Binary Search	LeetCode 704	Binary search (interpolation not needed)	Easy
Search Insert Position	LeetCode 35	Lower bound	Easy

Medium

Problem	Platform	Pattern	Difficulty
Find Peak Element	LeetCode 162	Binary search on derivative	Medium

Problem	Platform	Pattern	Difficulty
Search in Rotated Sorted Array	LeetCode 33	Rotated binary search	Medium

Hard

Problem	Platform	Pattern	Difficulty
Median of Two Sorted Arrays	LeetCode 4	Binary search	Hard
Kth Smallest Element in a Sorted Matrix	LeetCode 378	Binary search on value	Hard

18. Interview Explanation

"Interpolation search is a variant of binary search for sorted arrays. Instead of always checking the middle element, it estimates the position of the target using the formula $pos = low + (target - arr[low]) * (high - low) / (arr[high] - arr[low])$. This assumes the data is uniformly distributed. For uniformly distributed data, the average time complexity is $O(\log \log n)$, which is significantly faster than binary search's $O(\log n)$ for large arrays. However, for non-uniform distributions, the worst case is $O(n)$. In practice, binary search is more commonly used because it has guaranteed $O(\log n)$ performance regardless of the data distribution. Interpolation search is useful when you know the data is uniformly distributed and you need to minimize the number of comparisons."

19. Revision Notes

- **Key idea:** Estimate position using value-based interpolation.
- **Formula:** $pos = low + (target - arr[low]) * (high - low) / (arr[high] - arr[low])$.
- **Requires:** Sorted array, uniformly distributed values, arithmetic on values.
- **Complexity:** $O(\log \log n)$ average (uniform), $O(n)$ worst (non-uniform).
- **Space:** $O(1)$.
- **Main risk:** Division by zero (all equal), integer overflow.
- **Main advantage:** Very fast for large uniform arrays.
- **Main disadvantage:** Worst-case $O(n)$ for non-uniform data.
- **Common trap:** Using on non-uniform data, not handling division by zero.
- **Comparison:** Binary search ($O(\log n)$ always) is safer.

20. Final Cheat Sheet

Aspect	Detail
When to use	Sorted array, uniformly distributed values, large n
Formula	$pos = low + (target - arr[low]) * (high - low) / (arr[high] - arr[low])$
Time	$O(\log \log n)$ average, $O(n)$ worst
Space	$O(1)$
Key code	<pre>int pos = low + ((target - arr[low]) * (high - low)) / (arr[high] - arr[low]);</pre>

Aspect	Detail
Edge cases	Division by zero (all equal), overflow, out-of-bounds probe
Safety checks	<code>`if (pos < low</code>

End of Searching Algorithms Guide

This guide covers 9 essential searching algorithms for placements and competitive programming. Master binary search thoroughly — it is the most frequently asked searching algorithm in interviews. Practice lower/upper bound variants and search on answer patterns. The other algorithms (ternary, exponential, jump, interpolation) are less common but useful to know for specific scenarios.